



UNIVERSITY OF
CAMBRIDGE

Department of Computer
Science and Technology

Scheduling of Distributed LLM Serving on Heterogeneous GPUs

Nathan Rignall

Churchill College

June 2025

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Advanced Computer Science

Total page count: 63

Main chapters (excluding front-matter, references and appendix): 51 pages (pp 10–60)

Main chapters word count: 8576

Methodology used to generate that word count:

[For example:

```
$ make wordcount
gs -q -dSAFER -sDEVICE=txtwrite -o - \
    -dFirstPage=9 -dLastPage=52 report-submission.pdf | \
    egrep '[A-Za-z]{3}' | wc -w
8576
```

]

Declaration

I, Nathan Rignall of Churchill College, being a candidate for the Master of Philosophy in Advanced Computer Science, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed:

Date: 11/06/2025

Abstract

- LLM Inference Challenges.
- Separating Prefill and Decode phases.
- Challenges of Scheduling.
- MoE Scheduler is novel.
- Problem formulation (inner + outer).
- New Inner Loop.
- New Outer Loop.
- Results.

Acknowledgements

This project would not have been possible without the wonderful support of ... [optional]

Contents

1	Introduction	10
1.1	Motivation	10
1.2	Contributions	10
1.3	Outline	10
2	Background	11
2.1	Transformer Models	11
2.1.1	Computation Steps	12
2.1.2	Prefill and Decode	12
2.1.3	Key Value caching	13
2.1.4	Mixture of Experts	13
2.1.5	DeepSeek Architecture	13
2.2	Parallelism Strategies	13
2.3	Inference Challenges	14
2.4	Scheduler Technologies	15
2.4.1	Heuristic-Based Methods	15
2.4.2	Metaheuristic Algorithms	15
2.4.3	Search-Based	15
2.4.4	Constraint Programming	16
3	Related Work	17
3.1	Phase Splitting	17
3.1.1	Splitwise	18
3.1.2	DistServe	18
3.2	Heterogenous Serving	19
3.2.1	HexGen	19
3.2.2	Thunderserve	20
3.3	Summary and Future Work	20
4	Design and Implementation	21
4.1	Simulator	22
4.1.1	Shallowsim Implementation	22

4.1.2	Modifactions	23
4.1.3	Usage	23
4.2	Workload Modeling	24
4.2.1	Input Sequence Length	24
4.2.2	Decode Length	24
4.3	Objectives	26
4.3.1	Throughput Modelling	26
4.3.2	Throughput Allocation	27
4.4	Constraint Optimisation	28
4.4.1	Constraint Formulation	28
4.4.2	Challenges	29
4.5	Scheduling Assumptions	30
4.5.1	Prefill Parallelism	30
4.5.2	Decode Parallelism	31
4.5.3	Solver Resolution	33
4.6	Problem Formulation	36
4.7	Inner Loop	37
4.7.1	Constraint Formulation	38
4.7.2	Implementation	41
4.7.3	Optimisation	41
4.8	Outer Solver	42
4.8.1	Direct Island Generation	42
4.8.2	Divider Island Generation	44
5	Evaluation	48
5.1	Aims	48
5.2	System Performance	48
5.3	Inner Loop	49
5.3.1	Solver Results	50
5.3.2	Evaluator Results	52
5.3.3	Resolution Impact	52
5.3.4	Objective Impact	52
5.4	Outer Loop (Divider)	53
5.4.1	Batch Size Analysis	53
5.4.2	Skew Analysis	54
5.4.3	Number of Islands	54
5.5	Outer Loop (Direct)	55
5.6	Real-World	56
5.6.1	Traces	56
5.6.2	Results	58
5.7	Heterogeneous vs Homogenous	59

5.8	Discussion	59
6	Conclusion	60
6.1	Conclusion	60
6.2	Future Work	60

List of Figures

4.1	Sample probability distribution of input sequence lengths	24
4.2	Sample expected throughput	27
4.3	Sample throughput allocation mapping	27
4.4	Selection of GPUs benchmarked over the prefill stage using the <i>Shallowsim</i> simulator	30
4.5	Selection of GPUs benchmarked over the decode stage using the <i>Shallowsim</i> simulator	33
4.6	Impact of solver resolution on expected throughput	35
4.7	System Architecture	36
4.8	Diagram showing effective purpose of <i>inner loop</i>	37
4.9	Diagram or graph showing affect of caching	41
4.10	Diagram showing effective purpose of <i>outer loop</i>	42
4.11	Graph displaying the effect of changing the S_{skew} value	44
5.1	Benchmark results for every island test case	50
5.2	Assignment values percentage (per range) for each every island test case . .	51
5.3	Throughput per range for each every island test case	51
5.4	Inner Trace Throughput	52
5.5	Batch Size Analysis	53
5.6	Batch Size Analysis (Time)	53
5.7	Skew Analysis	54
5.8	Min Island Analysis	54
5.9	Direct SAASBO Batch Size Analysis	55
5.10	Direct SAASBO KSlots Size Analysis	55
5.11	Direct SAASBO Batch Size Analysis (Time)	55
5.12	Azure LLM Context and Generated Tokens	56
5.13	Azure LLM Context vs Generated Tokens	56
5.14	BurstGPT Context and Generated Tokens	57
5.15	BurstGPT Context vs Generated Tokens	57

List of Tables

5.1	GPU island test cases for <i>inner loop</i> analysis, with two homogeneous scenarios and two heterogeneous scenarios	49
-----	--	----

Chapter 1

Introduction

- Intro to Chapter.
- Inference for LLMs has energy and power requirements .
- Optimising this is important for the future.
- Heterogenous is better.
- Challenge is scheduling, that is what we look into.
- Specifically MoE.

1.1 Motivation

Stuff.

Questions?

1.2 Contributions

Lienar optimiser

Beysian optimisation

MoE integration

1.3 Outline

Chapter by chapter breakdown.

Chapter 2

Background

Within this Chapter we introduce the concepts behind transformer models, their associated computing steps and the different phases of inference. We also introduce several technologies which can be used for building schedules in computer systems.

2.1 Transformer Models

Transformer models are a type of deep learning architecture specifically designed to handle sequential data such as natural language. Unlike recurrent neural networks (RNNs) which process sequences one step at a time, transformers can analyse entire sequences of data in parallel. This is achieved through self-attention, where each token in a sequence can weigh and incorporate information from all other tokens. Such attention is calculated by adding positional encodings to input embeddings which transforms each token into query, key and value (KV) vectors [1].

The first transformer models were designed for encoder-decoder tasks such as translation, where an input sequence is encoded and decoded to an output sequence [2]. The encoder processes the entire input sequence and produces contextualised hidden representations of the text. Subsequently, the decoder generates output tokens one by one, using masked self-attention to prevent access to future tokens during generation [1]. Encoder-only models are typically used for classification and sentence embeddings [3], whereas decoder-only are often used for autoregressive generation (chatbots, coding assistants) [4].

Transformers form the basis of modern natural language processing due to their scalability, parallelisation benefits, and excellent performance on long-range dependencies. An LLM, or Large Language Model, is a type of transformer model that has been trained on a massive amount of text data. Their “large” size refers to the vast number of parameters they contain, which is key factor in their ability to capture complex patterns in natural language.

2.1.1 Computation Steps

There are several computation steps involved in a Transformer model’s forward pass during inference. The process begins with input embeddings and positional encodings, where each token in the input sequence is converted into a dense vector using a learned embedding matrix. Positional encodings are then added to incorporate information about the position of each token in the sequence. Next, the model computes the Query (Q), Key (K), and Value (V) vectors for each token by projecting the input embeddings through learned linear transformations. Using the Q, K, and V vectors, the model performs scaled dot-product attention to compute attention scores and weigh the significance of other tokens in the sequence relative to each token.

The output of the attention mechanism is passed through a residual connection and a layer normalisation step. This is followed by a position-wise feed-forward network, which consists of two linear transformations with a ReLU activation (or GELU in some models) in between, applied independently to each position. In deep Transformer models, these components (attention, residual connection, normalisation, and feed-forward layers) are stacked multiple times. The output of one layer serves as the input to the next. Finally, the output of the last Transformer layer is passed through a linear projection layer to produce the final output, depending on the task.

2.1.2 Prefill and Decode

We can separate the processing phases of LLM model inference steps into two distinct categories, with clear differences in hardware processing requirements.

The prefill stage processes the input tokens to compute the transformer model’s intermediate states. This includes calculating the positional embeddings and populating the KV cache, which are subsequently used to generate the first new token [5]. This specific phase is easily parallelised because at every stage of computation (per token) the full input sequence is known. Hence, this is viewed as a matrix-matrix operation which can easily saturate a GPU’s available compute [6]. *In reference to a transformer’s computation steps, all stages are performed for every token in the sequence but can be heavily parallelised.*

The decode phase generates each output token one at a time, this is because each token computation is dependent on the previous token [5]. Hence, each token must be calculated sequentially. The speed at which the decode phase can execute is heavily dependent on the speed at which weights, keys, values and activations can be loaded into the correct areas of GPU memory. This phase usually saturates the memory bandwidth of a GPU before saturating the compute performance [7]. *In reference to a transformer’s computation steps, all stages are performed, but for only the new token generated - this is due key-value caching.*

2.1.3 Key Value caching

Key value caching is an optimisation used during inference steps for transformer-based decoder-only models such as GPT [4]. This technique avoids recomputing the tensors for each input token for every decode token generated. Instead weights and key values are cached in GPU memory, to be used in the next iteration. At each decoding step the Q value of the new token is computed, reusing cached K and V values from earlier tokens in the sequence [8]. This technique comes at the cost of using more GPU memory, reducing the maximum model size that theoretically could be loaded.

2.1.4 Mixture of Experts

Mixture of Experts (MoE) is a neural network architecture designed to scale model capacity without a proportional increase in computational cost [9]. Unlike dense transformers where all parameters are used for every input, MoE models contain a pool of expert sub-networks, of which only a small subset are activated during inference for any given input token.

In a typical MoE transformer layer, a router network determines which experts to activate for each token. This is often achieved using top-k routing, where the router selects the top k experts based on a learned score for each input token. The selected experts are then applied to the input, and their outputs are combined before passing to the next layer [10].

2.1.5 DeepSeek Architecture

DeepSeek V3 and R1 are both LLMs that share a decoder-only Mixture-of-Experts (MoE) architecture with sparse expert activation, enabling over 670B total parameters while activating only approximately 37B per token. Both models support long-context inference (up to 128K tokens) and are trained using FP8 mixed-precision for efficiency.

A key feature of their model architecture is the use of KV cache compression during inference. Instead of storing full-precision key and value tensors for each expert, DeepSeek compresses these representations, significantly reducing memory bandwidth and enabling faster decode performance despite the model’s massive scale. V3 is tuned for general conversation, while R1 builds on it with enhanced reasoning capabilities.

2.2 Parallelism Strategies

Data parallelism refers to replicating an entire model across multiple processing units, with each replica receiving a different slice of the input data. Hence, this technique is generally used to increase throughput of an overall system, especially in batched inference scenarios.

Model parallelism involves partitioning the model itself across several processing units, where each device is responsible for a portion of the model’s layers or operations. This is most helpful when a model cannot fit on a single device, due to memory demand greater than what is available.

Tensor parallelism is a specialised form of model parallelism, generally used with large deep learning models. Individual tensors are split across multiple devices (rather than whole layers), reducing the communication overhead which may otherwise negatively impact latency.

Pipeline Parallelism is another form of model parallelism, where models are divided layer-wise (sequentially) and inputs are processed in a pipeline. This is in contrast to tensor parallelism, which are split within layers and model parallelism which is more coarse grained.

Expert Parallelism occurs when different experts are placed on different processing units and are activated in parallel. Some large language models use Mixture of Experts to help improve model performance.

Batching groups multiple inference requests to process them simultaneously, maximising hardware utilisation and overall system throughput.

Static batching accumulates requests until a batch is full or a timeout is reached. All requests are processed together and must wait for the slowest one to finish, leading to inefficient GPU usage and higher latency. *Continuous batching* is a dynamic method that immediately adds new requests to the batch as soon as others complete. This keeps the hardware constantly active, significantly increasing throughput and reducing idle time.

Batching increases throughput at the cost of individual request latency. Continuous batching is superior as it offers much higher throughput while also providing lower and more predictable latency

2.3 Inference Challenges

Inference for LLMs poses significant technical and operational challenges, especially at scale. Unlike traditional ML model, where inference is relatively lightweight, LLMs often require billions of parameters to be loaded and operated on for every user interaction. This results in high compute demands and considerable memory pressure, necessitating powerful and costly GPU infrastructure.

LLM Inference workloads also vary significantly depending on the application. For example, code generation workloads (e.g., copilots, IDE assistants) tend to have longer sequences and larger context windows, leading to slower decode-heavy patterns with less reuse and more frequent KV cache growth. In contrast, conversational workloads (e.g., chatbots) often involve shorter prompts and more frequent interactions, allowing for more

batching and reuse, but with higher real-time latency sensitivity. Understanding and modelling these patterns is critical for system-level optimisation and for deploying LLMs in a cost-efficient, scalable manner.

2.4 Scheduler Technologies

Context on why talking about schedulers...

Optimisation techniques...

2.4.1 Heuristic-Based Methods

What is this kind of technique?

First-Come, First-Served (FCFS) is...

Round Robin (RR) is...

Min-Min / Max-Min is...

Priority-Based is...

Load Balancing (Best Fit, Worst Fit) is...

2.4.2 Metaheuristic Algorithms

An intro...

Genetic Algorithms (GA) is...

Particle Swarm Optimization (PSO) is...

Ant Colony Optimisation (ACO) is...

Simulated Annealing (SA) is...

2.4.3 Search-Based

Search-based Optimisation refers to methods that systematically or semi-randomly explore a given search space to find an optimal solution. Such techniques are often used to tune a scheduler's parameters, but can also

Bayesian Optimisation (BO) Acquistition function Approximate function

Random Search Sample search space uniformly at random + evaluate Context - randomly generate candidate schedules, and pick the best one found Search space is significant

Grid Search Brute force Discrete set of possible values for each parameter, then evaluates every combination Suffers from curse of dimensionality

2.4.4 Constraint Programming

Both Constraint Programming and Mathematical Optimisation approaches formulate scheduling activities as an optimisation problem, with formal mathematical constraints and objectives.

Linear Programming (LP) techniques can solve scheduler problems where the constraints and objectives can be modelled as only linear. All decision variables are continuous and can be used in scheduling algorithms when fractional assignments of work are valid.

Mixed Integer Linear Programming (MILP) solvers can consider integer variables such as binary assignment values, extending LP. In the context of scheduling, this enables decisions to be modelled using yes/no decisions and combinatorial choices.

In the case where constraints cannot be as linear, **Non-Linear Programming (NLP)** can model problems where either the objective and/or constraints are non-linear functions. Given problems are non-linear, it is harder to guarantee a globally optimal solution is generated.

Mixed Integer Programming (MIP) is...

Chapter 3

Related Work

In this chapter we introduce the current state-of-the-art for large language model (LLM) inference techniques. We describe how *phase splitting* is currently being used to improve GPU cluster efficiency and reduce inference costs. Additionally, we describe how new advanced schedulers can be used to employ *phase splitting* in a heterogeneous environment to further optimise systems. Finally, we outline gaps in current work relating to heterogeneous *phase splitting* in mixture-of-experts (MoE) environments.

3.1 Phase Splitting

Previous works from Zhong *et al.* [11] and Patel *et al.* [7] introduced the concepts of *phase splitting* LLM requests into prefill and decode. Phase splitting is a technique that separates the two distinct computational stages of LLM inference, as described in subsection 2.1.2, onto separate hardware resources. This technique targets improving the overall cost efficiency of LLM inference, by improving the utilisation of hardware.

Running both compute phases on the same physical hardware often leads to inconsistent end-to-end latencies [7]. This is because there is typically strong prefill-decode interference as a result of batching, where each phase waits for the other to complete (to compete for GPU resources) [11]. To solve these latency inconsistencies, service providers have resulted to overprovisioning resources to meet service level objectives (SLOs) [11]. However, this results in sub-optimal use of hardware resources where available compute is often underutilised. This problem further worsens when the compute requirements for prefill and decode do not match each other, which is often the case in real-world scenarios.

Also, executing both prefill and decode on the same hardware limits the possible deployment scenarios of models, as prefill and decode have to use the same parallelism configurations [11]. Different parallelism techniques may be more optimal for prefill as compared to decode and visa versa. Hence, phase splitting provides important efficiency gains for LLM inference while demand from users is ever increasing.

3.1.1 Splitwise

Splitwise [7] is one of the first systems that formally introduced and analysed the benefits of phase splitting. The authors performed extensive analysis of LLM inference, characterising the prefill/prompt phase as compute-intensive and the decode/token phase as memory-intensive.

Splitwise divides the two inference phases onto separate machines using a hierarchical scheduling system. A Cluster-Level Scheduler (CLS) is responsible for machine pool management and for routing incoming requests to a prefill and decode machines. Pools are crafted using the analysed workload distribution (proportion of prefill vs decode) from public Azure LLM traces [7]. A Machine-Level Scheduler (MLS) then runs on each individual machine to manage its local request queue, making dynamic batching decisions.

During the initial prefill phase, the model processes the input prompt and populates the (KV) cache. This cache is required for the decode phase and because Splitwise runs the prefill and decode phases on different machines, the KV-cache generated on the prefill machine must be transferred over the network to the decode machine. To minimise latency, Splitwise overlaps the KV-cache transfer with the prefill computation.

The authors perform evaluation of Splitwise on NVIDIA A100s and H100s virtual machines on Microsoft Azure. The evaluation demonstrates phase splitting brings substantial improvements in throughput, cost, and power efficiency without compromising on latency SLOs. However, this work relies on high-end datacenter interconnects for its primary KV-cache transfer to function effectively, which may not always be available.

3.1.2 DistServe

Similar to Splitwise, DistServe [11] assigns the prefill and decoding phases to separate, dedicated machines, eliminating interference and allowing for independent optimisation. The authors optimise the system for "goodput", the maximum request rate that can be served whilst still adhering to defined SLOs.

Zhong *et al.* [11] introduce an end-to-end distributed servicing system which includes a placement algorithm, orchestration system and parallel execution engine. The scheduling algorithm maximises per-GPU goodput by first optimising the configuration for a single model replica and then scaling this configuration to meet the required traffic rate. During online operation, DistServe uses a simple First-Come-First-Served (FCFS) scheduling policy, where incoming requests are sent to the prefill instance with the shortest queue. Subsequent decode operations are sent to the least-loaded decoding instance.

The paper evaluate the DistServe system in a homogenous environment, with 4 nodes containing 8 NVIDIA SXM A100 connected with NVLINK. Using public datasets [12] [13] [14], the authors compare their system against vLLM [15] and DeepSpeed [16] inference

systems. The authors claim DistServe can serve up to 7.4x more requests or handle 12.6x tighter SLOs compared to state-of-the-art systems.

Like Splitwise, DistServe assumes the availability of high-quality, low-latency datacenter interconnects like NVLink for (KV) cache transfer. Hence, this system is less applicable to environments with constrained networking hardware. However, the authors do consider how parallelism strategies can be optimised for the prefill and decode phase independently, unlike Splitwise.

3.2 Heterogenous Serving

Building upon the concepts introduced with *phase splitting*, systems started to employ advanced scheduling methods to separate the prefill and decode phases onto specialised hardware. As the prefill and decode phase have different compute requirements it makes sense to target the workload deployment onto the most optimal hardware for the task. This is particularly relevant in heterogeneous environments such as the cloud, where a diverse mix of GPUs with varying costs, compute power, and memory bandwidths are available.

The schedulers introduced with Splitwise and DistServe both operate over a very small search space (due to the homogenous hardware). Hence, Jiang *et al.* [17] and Jiang *et al.* [18] work to develop more advanced scheduling algorithms, which optimise the time-to-solution and overall solution quality. The core challenge with heterogenous serving is not with the phase splitting itself, but with the scheduling algorithm used to organise the cluster

3.2.1 HexGen

HexGen [18] is a distributed inference engine designed to serve LLMs over heterogeneous environments, aiming to mitigate the substantial costs associated with traditional centralised deployments. The authors use a two-phase scheduling algorithm that solves the NP-hard scheduling problem.

The global optimisation processes uses a genetic algorithm to find the optimal partition of the global set of GPUs into multiple independent inference pipeline groups. This evolutionary algorithm gradually improves the solution by iteratively applying mutation operations such as Merge, Split, and Swap to the GPU partitions, which allows the search to explore different configurations and converge on a high-quality layout. The inner pipeline optimisation process uses dynamic programming to determine the optimal layout of pipeline stages.

A unique contribution of HexGen is its support for asymmetric parallelism, meaning each pipeline stage can be assigned a different tensor model parallel degree and number

of layers. This flexibility enables mapping of LLMs onto diverse GPUs with varying compute capabilities.

does not introduce a specific novel technique for KV cache management.

The authors claim that for the same budget, HexGen can achieve up to 2.3x lower latency deadlines or tolerate up to 4x more requests than a homogeneous baseline.

3.2.2 Thunderserve

Jiang *et al.* [17] build Thunderserve, another LLM serving system designed specifically for heterogeneous hardware. The authors use a novel two-level hierarchical scheduling algorithm to optimise the deployment plan across varied hardware and network conditions.

The global/upper level uses a search algorithm to determine the optimal group construction (how to partition GPUs) and phase designation (assigning groups to prefill or decode). Whereas the lower level, given the group assignments, deduces the optimal parallel configuration and orchestrates request routing by framing it as a solvable transportation problem. The authors also introduce lightweight re-scheduling mechanisms, designed to adapt to fluctuating online conditions like workload shifts or node failures without service restarts. The system adapts to changing workloads by only adjusting phase designation and orchestration, to minimise user disruption from rescheduling.

Good cost efficiency

The KV cache compression is a lossy process that could negatively affect performance on tasks requiring perfect fidelity (additional analysis required)

Also the search algorithm Search heuristic to solve an NP-hard problem; while effective, it is not guaranteed to find the absolute global optimum,

3.3 Summary and Future Work

The initial works of DistServe and Splitwise established the initial concepts of phase splitting to improve LLM inference efficiency in primarily homogeneous settings. Building upon this, HexGen and ThunderServe developed more advanced schedulers to apply these principles to complex, heterogeneous cloud environments. HexGen introduced asymmetric parallelism for greater flexibility, while ThunderServe focused on holistic deployment optimisation and dynamic adaptation for the cloud.

A shared limitation across these systems is that they primarily focus on standard transformer architectures. They do not explicitly consider or optimise for the unique characteristics of Mixture-of-Experts (MoE) models, which have different architectural and computational patterns. We exploit this gap to develop a heterogeneous scheduler for MoE, targeted towards the state-of-the-art DeepSeek V3 architecture [19].

Chapter 4

Design and Implementation

- Intro to Chapter.
- Explain GPU islands.
- Scheduler traffic direction (architecture).
- GPU Configurations.

4.1 Simulator

In order to be able to develop an advanced heterogenous LLM scheduler, we need access to many high-performance GPUs to test our schedule configurations. Unfortunately, GPU-hours are prohibitively expensive and using such hours for “dummy” workloads to test a scheduler would be considered a waste of resources. Instead, we utilise the latest simulator technology to estimate the performance of LLM inference given a set of input parameters. Several simulators exist that attempt to model the performance of an LLM request by using estimation techniques.

Vidur [20] (essentially uses linear regression)

Shallowsim [21] is a specialist simulator designed to estimate the compute times for different LLM request types.

LLMCompass [22] (more detail)

We Select Shallowsim for its transparency for calculations (simplicity). And support for Mixture of Expert models (key novel).

4.1.1 Shallowsim Implementation

Prefill

Dense-MLA (dense Multi-head/Multi-query Attention compute),

TP-MLA (Dense-MLA plus the Tensor-Parallel all-reduce),

Dense-MLP (feed-forward network compute),

Shared-Expert (MoE expert run that every token visits), Routed-Expert (MoE expert run that only selected tokens visit),

Every token first passes through the Shared Expert, but a subset of tokens are also routed to extra specialists. This is modelled with the Routed Expert function.

Dispatch (All-to-All that ships tokens out to remote experts), Combine (All-to-All that brings tokens back),

Sending activations to the respective routed expert nodes is modelled using the Dispatch step, the Combine step then models how processed activations are collated back together. These two functions model the network overhead of LLM queries.

Compute Time - Dense MLA, Dense MLP (Dense Layers) - TP-MLA, Shared Expert, Routed Expert (Sparse Layers) Communication Timemake - Dispatch, Combine (Sparse Layers) Overlap Penalties - If Combine is longer than TP-MLA + Shared Expert - If Dispatch is longer than Routed Expert Sum - Compute + Communication + Any Overlap = Prefill Time

Decode

Load-KV, Dense-MLA (as prefill), Sparse-MLA, Dense-MLP (as prefill), Shared-Expert (as prefill), Routed-Expert (as prefill), Dispatch (as prefill), Combine (as prefill),

Compute Time - Dense MLA, Dense MLP (Dense Layers) - Sparse MLA, Shared Expert, Routed Expert (Sparse Layers) Communication Time - Dispatch, Combine (Sparse Layers) Overlap Penalties - TODO Sum - TODO = Per token decode time

4.1.2 Modifactions

- What we have done to modify the shallowsim simulator.
- As designed, shallowsim is build for profiling several different GPU configurations at once.
- For our system, we are particularly interested in per-configuration results.

4.1.3 Usage

Prefill Phase

Using the Shallowsim simulator we can measure the time t_{ms} (in milliseconds) to compute a single prefill request for a given set of parameters, and convert this to throughput R (in requests per second) via $R_{\text{prefill}} = \frac{1000}{t_{\text{ms}}}$.

Decode Phase

Shallowsim has support for calculating the time in milliseconds for a single token generation step. As we would typically expect several tokens to be generated, we cannot directly compute requests per second using Shallowsim, instead we must perform additional computations. The decode time of a single token is affected by the input/current sequence length, therefore we must measure the single token compute time for every token to be generated.

By measuring $t_{\text{ms},i}$ the compute time (in milliseconds) for the i th token, the total decode time for a request generating N tokens is $t_{\text{decode}} = \sum_{i=1}^N t_{\text{ms},i}$.

We can then convert this total time into throughput R_{decode} (in requests per second), using the same method as prefill. $R_{\text{decode}} = \frac{1000}{t_{\text{decode}}} = \frac{1000}{\sum_{i=1}^N t_{\text{ms},i}}$, where R_{decode} represents how many full decode requests per second the configuration can sustain.

4.2 Workload Modeling

We expect LLM inference services to encounter several different classes of workloads, depending on the user’s use-case. These workloads have different characteristics of input sequence length (request message plus any system prompts), and also different decode lengths (numbers of characteristics generated). For example, one would expect code prompts and responses to have different input sequence and decode lengths than to that of conversation prompts. Hence, we must design a scheduler that is able to optimise over a specific workload class.

4.2.1 Input Sequence Length

To achieve fine-grained control of request scheduling, dependant on input sequence, we model the input request length as a probability distribution. Hence, for a chosen workload type, there is a specific probability of request arriving with given sequence length.

Let $w \in W$ denote a workload type from the set of all workload types W . For each workload type w , we define a probability mass function $p(s | w)$, which represents the probability of a request having sequence length $s \in S$ under workload w . This distribution satisfies: $\sum_{s \in S} p(s | w) = 1$ for all $w \in W$.

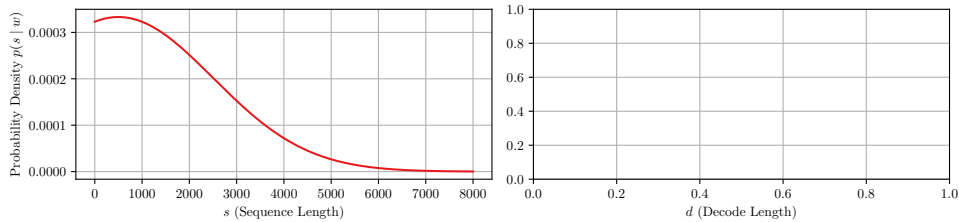


Figure 4.1: Sample probability distribution of input sequence length. Modelled using a pdf where $\mu = 500$, $\sigma = 2000$ and truncated/normalised to the range $0 \leq s \leq 8000$.

Instead of only considering the distribution of sequence lengths during assignment, to model system throughput, we use them to find the most optimal pairing of sequence length to GPU island. Therefore, specific GPU islands can be configured to most optimally perform prefill operations for a specific sequence length. When a request arrives, we can direct it to the GPU islands targeted to serve that request length, defined in the schedule.

4.2.2 Decode Length

While decode length can also be modelled using a probability density function, this information (specific distribution) is less useful from a scheduling perspective. In contrast to input sequence length, the decode length is not known the moment a request arrives. Instead, the generation process proceeds token-by-token until an explicit or implicit stop condition is met (e.g. end-of-sequence token).

We can assume that a decode request must complete on the GPU island it started processing on, as switching island would incur additional communication overhead and significantly impact the request response time.

For some workloads there is a small correlation between input sequence length and decode length, but this relationship is often not significant. Therefore, at the point of request arrival we do not have enough information about the decode length to make informed per-decode length scheduling decisions. Hence, creating specific GPU configurations for short, medium, long, etc. decode lengths would likely be inefficient. As we cannot accurately predict decode length, there is a high probability that a decode request would have to move island.

Instead, we model our scheduler using an average decode length for a given workload type and design every GPU decode island to support the maximum decode length expected. Unlike the sequence length, the decode length is simply being used for system modelling activities to characterise performance, not direct traffic. However, we can still make scheduling decisions for the decode phase using the sequence length as a feature, given the maximum decode length affected by the input length. This is due to all the input and decode tokens KV values being stored in GPU memory.

4.3 Objectives

4.3.1 Throughput Modelling

Demonstrated in subsection 4.1.3, the Shallowsim simulator has support for estimating the throughput of a single request of a static input sequence and decode length. It is highly improbable that an entire workload consists of a single request type, instead we expect a distribution of sequence lengths, as shown in section 4.2. Hence, we need a method to accurately calculate the throughput of a system, with a variety of input sequence.

Using the probability distribution of the workload, we can compute the weighted average of throughput, using measured performance values for each sequence length. Let $R_{\text{prefill}}(s)$ denote the measured prefill throughput (in requests per second) for a request of sequence length s . The expected prefill throughput for workload w , denoted $\mathbb{E}[R_{\text{prefill}} | w]$ (expectation), is then given by:

$$\mathbb{E}[R_{\text{prefill}} | w] = \sum_{s \in S} R_{\text{prefill}}(s) \cdot p(s | w)$$

In addition to prefill, we can also compute the expected throughput for the decode phase. While prefill throughput depends solely on the input sequence length s , decode throughput typically depends on both the input length s and the output (decode) length d . For simplicity, we assume a fixed average decode length d as a parameter. Let $R_{\text{decode}}(s, d)$ denote the measured decode throughput (in requests per second) for an input sequence length s and average decode length d . Given a workload-specific sequence length distribution $p(s | w)$, the expected decode throughput for workload w is:

$$\mathbb{E}[R_{\text{decode}} | w, d] = \sum_{s \in S} R_{\text{decode}}(s, d) \cdot p(s | w)$$

To account for the complete request processing pipeline, we define the overall expected throughput as the minimum of the expected prefill and decode throughputs. Since requests cannot be served faster than the slowest stage, the effective throughput is bounded by the bottleneck. Let $\mathbb{E}[R_{\text{prefill}} | w]$ denote the expected prefill throughput for workload w , and $\mathbb{E}[R_{\text{decode}} | w, d]$ the expected decode throughput for workload w and average decode length d . The overall expected throughput is given by:

$$\mathbb{E}[R_{\text{overall}} | w, d] = \min(\mathbb{E}[R_{\text{prefill}} | w], \mathbb{E}[R_{\text{decode}} | w, d])$$

This value represents the effective throughput achievable by the system under the given workload and output length assumptions.

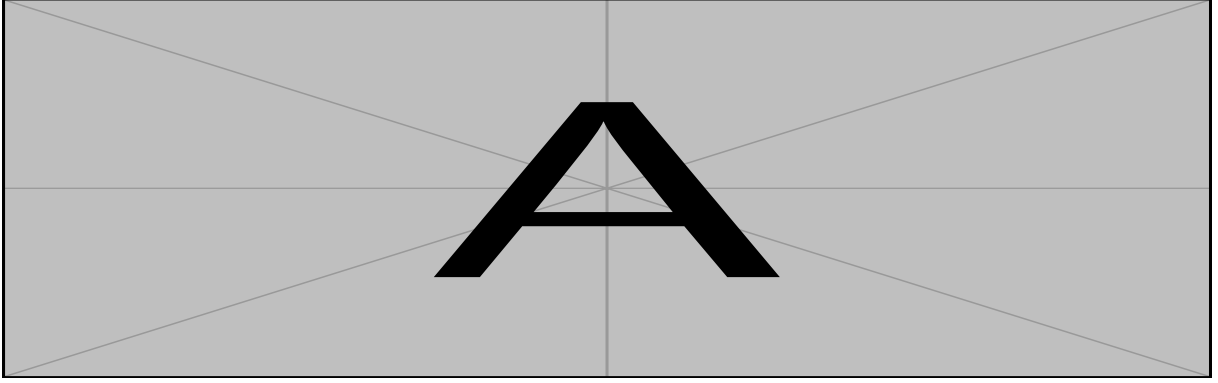


Figure 4.2: Sample expected throughput, calculated using the workload shown in Figure 4.1.

4.3.2 Throughput Allocation

Previous calculations assumed that a singular GPU island is contributing to the overall throughput of the system. In reality, we would like to be able to construct several GPU islands and assign each island to a request type it is best suited for. Hence, we allocate a percentage of GPU resources for a given island to a given sequence length it specialises in. Let I be the set of all GPU islands in the system, which we partition into two separate subsets I_{prefill} and I_{decode} . Let $x_i(s)$ denote the fraction of overall GPU capacity on island i devoted to handling requests of length s . We normalise these allocations so that $\sum_{s \in S} x_i(s) = 1$ for every $i \in I$. By including $x_i(s)$ into our previous per-length throughput models, the expected prefill throughput under workload w becomes

$$\mathbb{E}[R_{\text{prefill}} \mid w] = \sum_{i \in I_{\text{prefill}}} \sum_{s \in S} x_i(s) R_{\text{prefill}}^{(i)}(s) p(s \mid w),$$

while the expected decode throughput for average decode length d is

$$\mathbb{E}[R_{\text{decode}} \mid w, d] = \sum_{i \in I_{\text{decode}}} \sum_{s \in S} x_i(s) R_{\text{decode}}^{(i)}(s, d) p(s \mid w).$$

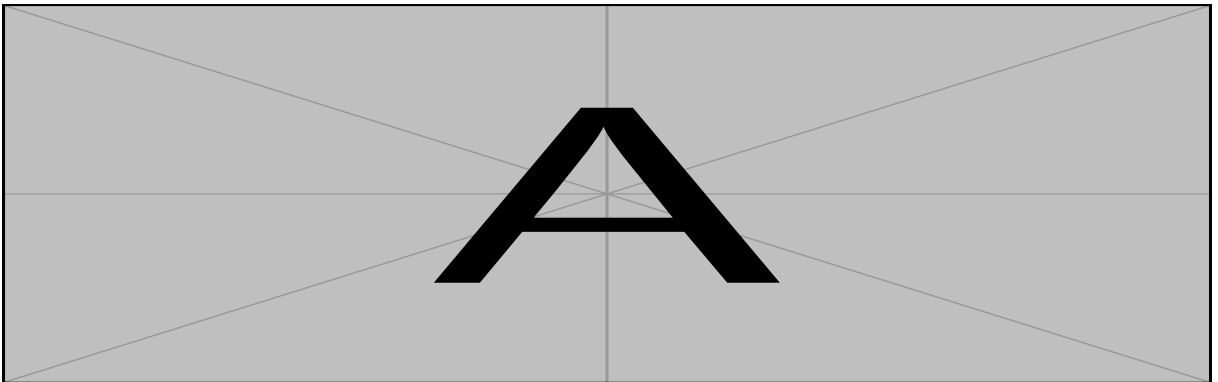


Figure 4.3: Sample throughput allocation mapping.

4.4 Constraint Optimisation

We can combine the previous throughput calculations to form a simple constraint optimisation problem with an objective to maximise the total expected throughput R and minimise the deviation Δ from the target workload distribution $p(s | w)$.

4.4.1 Constraint Formulation

The following statement describes a sample non-linear optimisation problem that can be solved for either prefill or decode throughput.

Input Data

- I (set of GPU islands),
- S (set of sequence lengths),
- $b_i(s)$ (benchmark throughput of island i on length s),
- $p(s | w)$ (target probability of length s under workload w), $\sum_{s \in S} p(s | w) = 1$,
- λ (trade-off parameter, $\lambda > 0$, balancing throughput vs. deviation).

Decision Variables

$$x_i(s) \geq 0 \quad (\text{fraction of island } i\text{'s GPU capacity devoted to length } s).$$

Constraints

$$\sum_{s \in S} x_i(s) = 1 \quad \forall i \in I.$$

Intermediate Quantities

- Performance per cell:

$$r_i(s) = b_i(s) x_i(s).$$

- Total performance:

$$R = \sum_{i \in I} \sum_{s \in S} r_i(s) = \sum_{i \in I} \sum_{s \in S} b_i(s) x_i(s).$$

- Performance by sequence length:

$$R_s = \sum_{i \in I} r_i(s) = \sum_{i \in I} b_i(s) x_i(s), \quad \forall s \in S.$$

- Achieved share:

$$\sigma_s = \frac{R_s}{R}.$$

- **Deviation from target:**

$$\delta_s = |\sigma_s - p(s \mid w)|, \quad \Delta = \sum_{s \in S} \delta_s.$$

Objective

Maximise

$$R - \lambda \Delta,$$

where

$$R = \sum_{i \in I} \sum_{s \in S} b_i(s) x_i(s), \quad \Delta = \sum_{s \in S} |\sigma_s - p(s \mid w)|, \quad \sigma_s = \frac{\sum_{i \in I} b_i(s) x_i(s)}{R}.$$

4.4.2 Challenges

The constraint optimisation problem described is non-linear, this is due to the chained multiplications and divisions for the *performance per cell* and *achieved share* calculations. As a result, we require a non-linear programming (NLP) solver to obtain a solution for the proposed problem.

While the problem can be solved using such NLP techniques, the process tends to be computationally intensive and slow. This is primarily due to the non-convex and non-linear nature of the formulation, which makes it difficult to guarantee convergence to the global optimum. Ideally, we would like to linearise this problem to improve computational efficiency and enhance the reliability of finding the best possible solution.

Furthermore, we statically define I to a set of curated islands, and the optimiser simply fits the workload to these islands. In order to generate the most optimal solution, we need to choose the best set of island sizes for each GPU type. However, adding this decision process to the constraint optimisation problem increases its complexity, as it effectively turns a bounded bin packing problem into an unbounded one. The optimiser must now determine not only the workload allocation but also the optimal sizes and number of islands. Hence, we need to formulate an architectural approach that incorporates this logic, which we detail in section 4.6.

4.5 Scheduling Assumptions

Before finalising our problem, we can make several assertions and assumptions in an attempt to reduce the search space of the scheduler generation process. These effectively apply heuristics to our problem, that are crafted to simplify the computation requirements while retaining solution quality.

4.5.1 Prefill Parallelism

We can assume for a given model and GPU type, there exists an optimal parallelism configuration that is consistent across input sequence lengths. Demonstrated in Figure 4.4 we can see that for all GPUs, $tp = 64$ and $dp = 1$ is the fastest parallelism configuration for all sequence lengths above 128 tokens. Given we expect the majority of input requests to be over 128 tokens, we can reason that there is an *optimal* GPU prefill configuration for each GPU type.

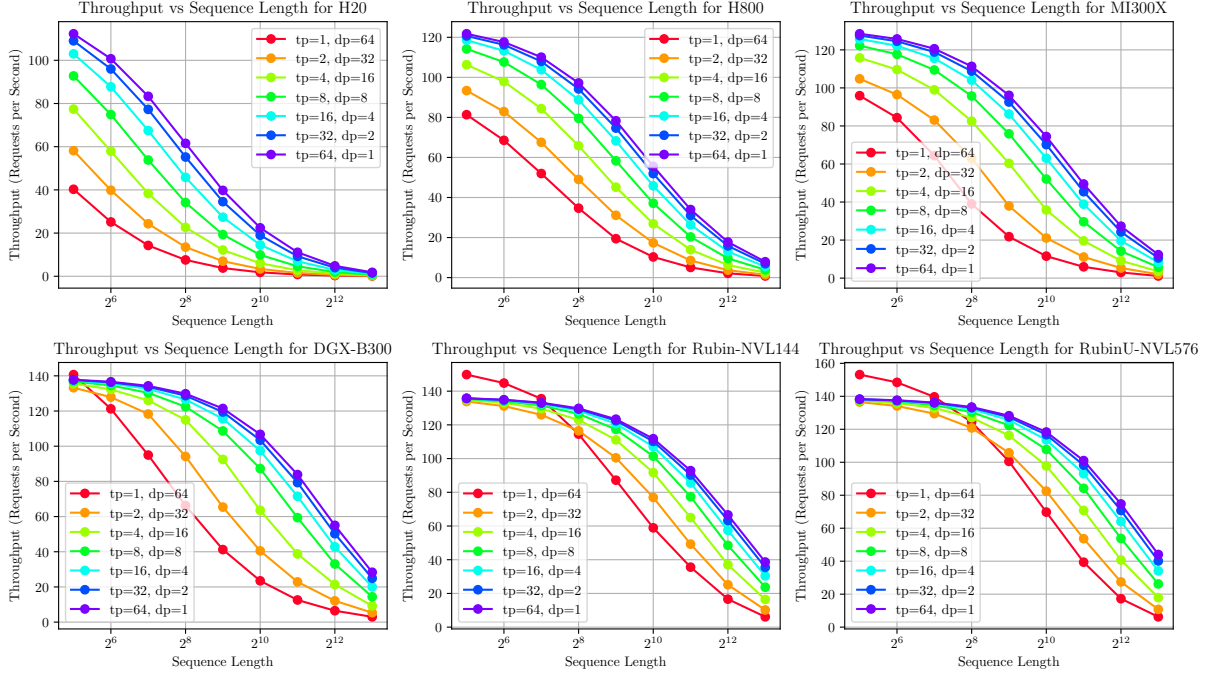


Figure 4.4: Selection of GPUs benchmarked over the prefill stage, with a variety of parallelism configurations using the *Shallowsim* simulator. All configurations use a static number of GPUs (64).

By pre-computing this optimal configuration, we significantly reduce the number of parameters considered by the scheduler. This is because an island cannot be formed with several different parallelism configurations simultaneously. And hence, the scheduler does not have to contain a variable for (and consider) each sequence-length-parallelism combination.

To identify a best GPU prefill configuration across sequence lengths we use the following calculations, let G be the total number of GPUs and consider the set of all parallelism

configurations $C = \{c = (tp, dp) \mid tp \cdot dp = G\}$. For each configuration $c \in C$ and sequence length $s \in S$, we measure the prefill throughput $R_{\text{prefill}}(s, c)$ and define the optimal throughput at length s by

$$R_{\text{prefill}}^*(s) = \max_{c \in C} R_{\text{prefill}}(s, c).$$

We then compute the relative deviation of configuration c at length s as

$$\delta(c, s) = \frac{R_{\text{prefill}}^*(s) - R_{\text{prefill}}(s, c)}{R_{\text{prefill}}^*(s)}.$$

Aggregating these deviations over all lengths gives

$$\begin{aligned} \bar{\delta}(c) &= \frac{1}{|S|} \sum_{s \in S} \delta(c, s), & (\text{uniform average over } S), \\ \tilde{\delta}(c) &= \sum_{s \in S} \delta(c, s) p(s \mid w), & (\text{weighted by workload distribution}). \end{aligned}$$

Finally, the best overall configuration is chosen as the one minimising the average deviation from the individual best configurations.

$$c_{\text{prefill}} = \arg \min_{c \in C} \tilde{\delta}(c).$$

4.5.2 Decode Parallelism

We can compute the same best configuration for the decode phase, with small modifications to the set of C configurations taking into account mixture-of-experts. Unlike prefill, dp (the data-parallelism factor) is not a native parameter of the Shallowsim simulator. Instead, we pre-compute how many GPUs to pass to the simulator and then scale the resultant throughput by dp . Let G be the total number of GPUs, tp tensor parallelism factor, and n the total number of routed experts in the model. We can form every pairing of valid factors using:

$$T = \{tp \in \{1, \dots, G\} \mid G \bmod tp = 0\}, \quad D = \{dp \in \{1, \dots, G\} \mid G \bmod dp = 0\},$$

$$C' = T \times D = \{(tp, dp) \mid tp \in T, dp \in D\}.$$

For each $(tp, dp) \in C'$, set

$$G_{\text{sim}} = \frac{G}{dp}, \quad ep = \left\lceil \frac{n}{G_{\text{sim}}} \right\rceil,$$

and define the simulator configuration

$$c = (G_{\text{sim}}, tp, ep) \in C.$$

Then, for each $c \in C$ and sequence length $s \in S$, the simulator returns

$$R_{\text{prefill}}(s; c),$$

and under dp -way data parallelism the effective total prefill throughput is

$$R_{\text{prefill}}(s; dp, c) = dp \times R_{\text{prefill}}(s; c) = dp \times R_{\text{prefill}}\left(s; \frac{G}{dp}, tp, ep\right).$$

Before calculating the best parallelism configuration out of the set, we also need to decide which batch size to use. Since we would like to optimise our configuration for overall throughput, we need to maximise the batch size for each input sequence length, such that we remain within the memory constraints of the island. Let $\max_bs(s; c)$ denote the maximum batch size returned by the simulator for sequence length s under configuration c . We then round down to the nearest multiple of 8 and clamp between 8 and 1024 to keep the batch size aligned to the GPU memory and compute capabilities. This ensures efficient hardware utilisation and avoids potential performance degradation caused by unaligned memory access or suboptimal parallelism.

$$bs(s; c) = \min\left(\max(\max_bs(s; c) - (\max_bs(s; c) \bmod 8), 8), 1024\right).$$

Similar to prefill, we seek a single best decode parallelism configuration across sequence lengths. For each simulator configuration $c \in C$ and sequence length s , we first compute the batch size $bs(s; c)$ as before, then measure the decode throughput $R_{\text{decode}}(s, d; c, bs(s; c))$. We define the optimal decode throughput at length s by

$$R_{\text{decode}}^*(s, d) = \max_{c \in C} R_{\text{decode}}(s, d; c, bs(s; c)),$$

and the relative deviation of configuration c at s by

$$\delta_{\text{decode}}(c, s) = \frac{R_{\text{decode}}^*(s, d) - R_{\text{decode}}(s, d; c, bs(s; c))}{R_{\text{decode}}^*(s, d)}.$$

Aggregating these deviations over all lengths gives

$$\begin{aligned} \bar{\delta}_{\text{decode}}(c) &= \frac{1}{|S|} \sum_{s \in S} \delta_{\text{decode}}(c, s), & (\text{uniform average over } S), \\ \tilde{\delta}_{\text{decode}}(c) &= \sum_{s \in S} \delta_{\text{decode}}(c, s) p(s | w), & (\text{weighted by workload distribution}). \end{aligned}$$

We can then select the decode configuration that minimises the average deviation across all expected sequence lengths,

$$c_{\text{decode}} = \arg \min_{c \in C} \tilde{\delta}_{\text{decode}}(c).$$

Shown in Figure 4.5, for each GPU there is generally a better parallelism configuration for most sequence lengths. Unlike prefill, this configuration is not consistent across GPU types, but rather significantly impacted by the GPU specifications. Configurations with zero throughput are due to the configuration exceeding the memory capabilities of the cluster.

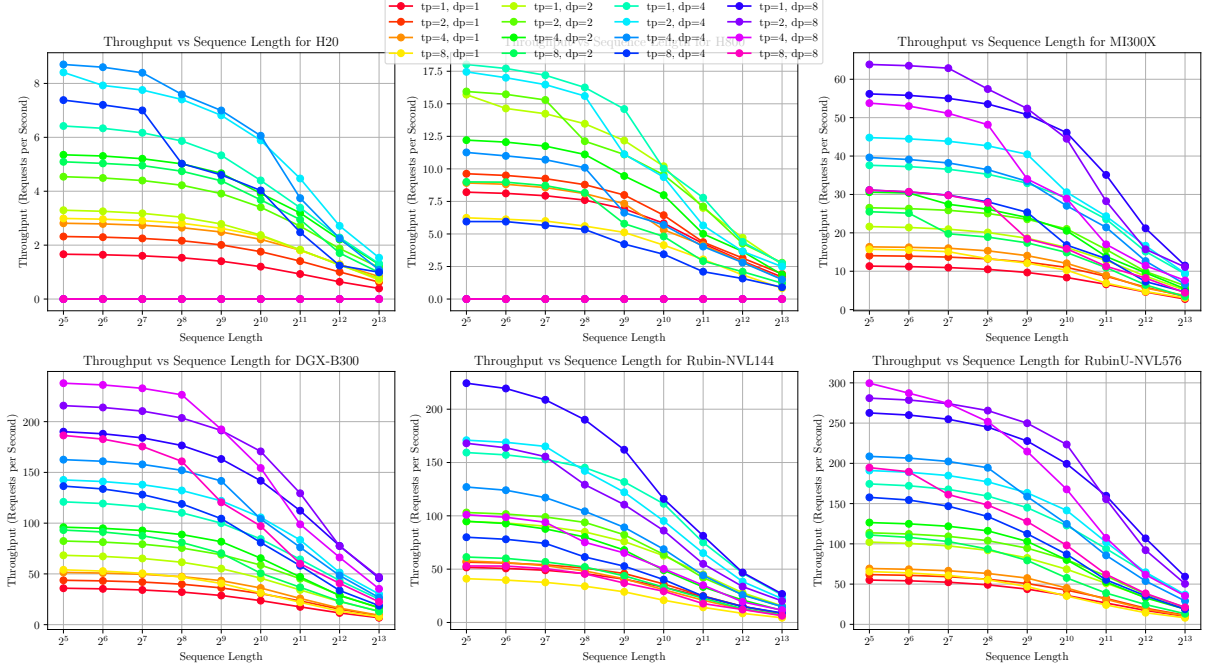


Figure 4.5: Selection of GPUs benchmarked over the decode stage, with a variety of parallelism configurations using the *Shallowsim* simulator. All configurations use a static number of GPUs (64).

4.5.3 Solver Resolution

We can use the simulator to benchmark the prefill and decode throughput of an island for a given input sequence length. While it is feasible to consider each sequence length ($s \in S$) individually (in our optimisation problem), we can use a binning approach to reduce the search space of the scheduler. The width of the ranges effectively control the resolution of the solution generated, which can be dynamically adjusted depending on input parameters.

We partition the sorted list of lengths S into equally-sized intervals of width W , called *ranges*. Rather than measure every s_i , we sample a single representative length from each range at its midpoint. With a positive integer width W , we compute the midpoint offset as $\text{mid} = \lfloor \frac{W}{2} \rfloor$. For each integer k such that $0 \leq k < \lfloor |S|/W \rfloor$, we let

$$\text{range_start}_k = kW, \quad \text{range_end}_k = kW + W, \quad \text{range_mid}_k = \text{range_start}_k + \text{mid},$$

and take range_mid_k as the representative length for the k -th range. We then also compute

the total probability of that range by summing

$$p_k = \sum_{i=\text{range_start}_k}^{\text{range_end}_k-1} p(s_i | w),$$

and record for range k the tuple $(\text{range_mid}_k, p_k, \text{range_start}_k, \text{range_end}_k)$. Therefore, the set of all ranges is

$$\mathcal{R} = \{ (\text{range_mid}_k, p_k, \text{range_start}_k, \text{range_end}_k) \mid 0 \leq k < \lfloor |S|/W \rfloor \}.$$

By sampling only at these midpoints and aggregating probabilities, we approximate the full set of sequence lengths in each range with a single point (range_mid_k) , thereby greatly reducing the number of simulator calls while preserving workload-weighted accuracy.

Prefill Throughput

With the formed ranges $\mathcal{R}$, each with their representative length s_r and total probability p_r , we can redefine the allocation variables $x_i(s)$ to act at the range level. Now island i assigns fraction $x_i(r)$ of its GPU capacity to range $r \in \mathcal{R}$, with

$$\sum_{r \in \mathcal{R}} x_i(r) = 1, \quad x_i(r) \geq 0 \quad \forall i \in I, r \in \mathcal{R}.$$

Within range r , every sequence length $s \in S_r$ is approximated by s_r , so that

$$\sum_{s \in S_r} R_{\text{prefill}}^{(i)}(s) p(s | w) \approx R_{\text{prefill}}^{(i)}(s_r) \sum_{s \in S_r} p(s | w) = R_{\text{prefill}}^{(i)}(s_r) p_r.$$

By including $x_i(r)$ into our per-range throughput models, the expected prefill throughput under workload w becomes

$$\mathbb{E}[R_{\text{prefill}} | w] = \sum_{i \in I_{\text{prefill}}} \sum_{r \in \mathcal{R}} x_i(r) R_{\text{prefill}}^{(i)}(s_r) p_r.$$

Decode Throughput

The same approach applies to the decode phase. For each range r with representative length s_r and probability p_r , we measure the decode throughput $R_{\text{decode}}^{(i)}(s_r, d)$ on island i for average decode length d . Island i allocates fraction $x_i(r)$ of its capacity to r , and within that range every sequence length is approximated by s_r . Thus,

$$\sum_{s \in S_r} R_{\text{decode}}^{(i)}(s, d) p(s | w) \approx R_{\text{decode}}^{(i)}(s_r, d) p_r.$$

Normalising $x_i(r)$ as before and summing over islands and ranges yields the expected decode throughput under workload w and decode length d :

$$\mathbb{E}[R_{\text{decode}} \mid w, d] = \sum_{i \in I_{\text{decode}}} \sum_{r \in \mathcal{R}} x_i(r) R_{\text{decode}}^{(i)}(s_r, d) p_r.$$

The overall expected throughput remains

$$\mathbb{E}[R_{\text{overall}} \mid w, d] = \min(\mathbb{E}[R_{\text{prefill}} \mid w], \mathbb{E}[R_{\text{decode}} \mid w, d]).$$

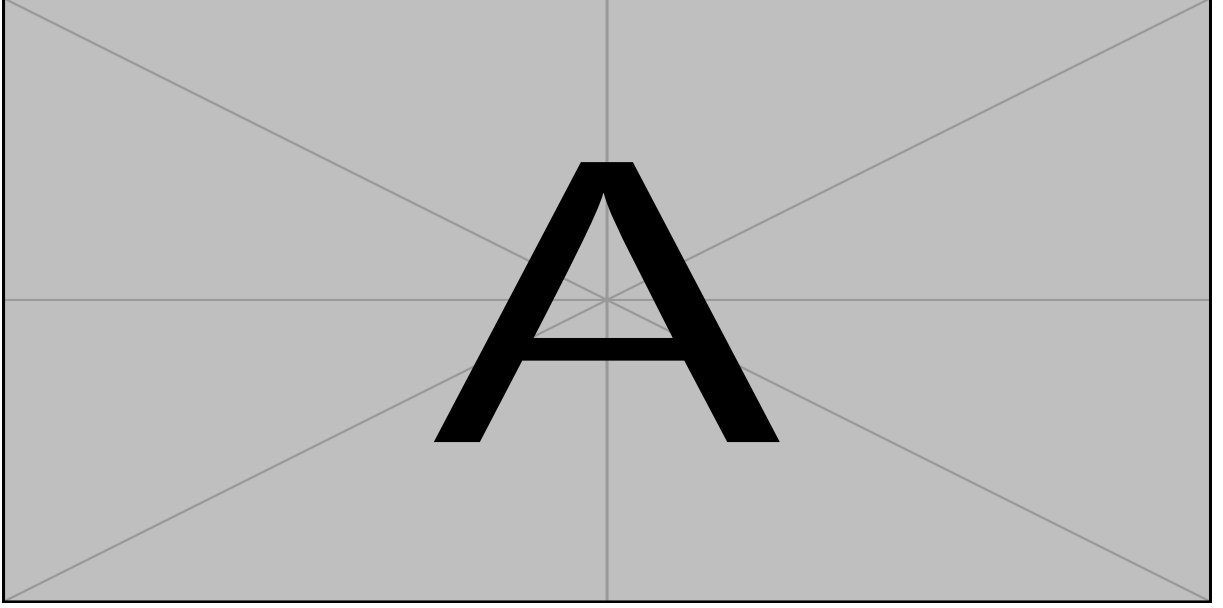


Figure 4.6: Impact of solver resolution on expected throughput, calculated using the workload shown in Figure 4.1.

4.6 Problem Formulation

During section 4.4 we formulated a non-linear bounded bin-packing problem, by constraining the islands to a pre-defined type and size. In a true scheduling problem we expect to be allocated an inventory of different GPU types, not formed islands. Let T be the set of GPU types and for each $t \in T$, let I_t be the number of GPUs of type t allocated in the inventory. The full GPU inventory is then the set $\{I_t : t \in T\}$, with total number of GPUs in the system modelled as $N = \sum_{t \in T} I_t$.

Similar to ThunderServe [17], we can separate this scheduling problem as two separate, distinct stages. The first stage is the island layout generation, which we can treat as a search-like optimisation problem in which we explore combinations of GPU island sizes and types. The second stage is the bounded bin-packing step, a deterministic problem that assigns workload ranges to the pre-computed islands to maximise throughput. By decoupling layout design from the packing phase, we can leverage specialised solvers for each subproblem and significantly reduce overall computational complexity.

We refer to the bounded bin packing problem as the *inner loop*, and the outer optimisation process as the *outer loop*, as illustrated in Figure 4.7. The *outer loop* is responsible for generating promising island layouts, which are then passed to the *inner loop* for resource packing. The resulting throughput is returned to the *outer loop*, allowing the optimisation process to evaluate and refine its search for effective configurations. This iterative cycle can be repeated multiple times to generate progressively generate an optimum solution.

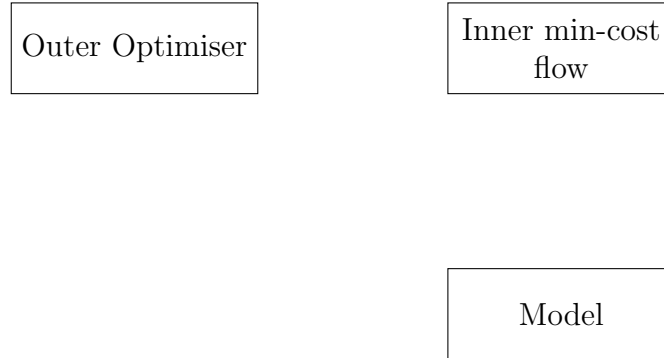


Figure 4.7: System Architecture

4.7 Inner Loop

The *inner loop* receives an island layout proposal from the *outer loop*, this layout is specified as a finite index set $\mathcal{I} = \{i = 1, \dots, |\mathcal{I}|\}$, where each island i is represented by a pair (t_i, s_i) . Here, $t_i \in T$ denotes the GPU type assigned to island i , and $s_i \in \mathbb{Z}_{\geq 0}$ indicates the number of GPUs of type t_i allocated to that island. The *outer loop* guarantees that these allocations respect the total-inventory constraints.

Given the predefined island layout $\{(t_j, s_j)\}_{j=1}^{|\mathcal{I}|}$, the inner loop's only task is to assign workloads of different prefill and decode lengths. No further modification of t_i or s_i takes place at this stage, as the inner loop focuses entirely on determining the optimal bin-packing configuration.

We can outline the steps of the *inner loop* as follows:

1. **Formulate Ranges.** Take the specific workload distribution w , and separate into a series of ranges $\mathcal{R}$ using a specified resolution (subsection 4.5.3)
2. **Prefill Configuration Generation & Benchmark.** Identify the best prefill configuration (c_{prefill}) for each island i , and return the benchmark results describing the throughput for each range (subsection 4.5.1).
3. **Decode Config Generation & Benchmark.** Identify the best decode configuration (c_{decode}) for each island i , and return the benchmark results describing the throughput for each range (subsection 4.5.2).
4. **Problem Specification.** Formulate the exact optimisation problem based on the provided prefill and decode benchmarks (subsection 4.7.1).
5. **Solve.** Invoke the optimiser on the specified problem, to generate the best solution given the input parameters.
6. **Results.** Extract the final workload-assignment solution from the solver's output, including any intermediary variables for debugging.

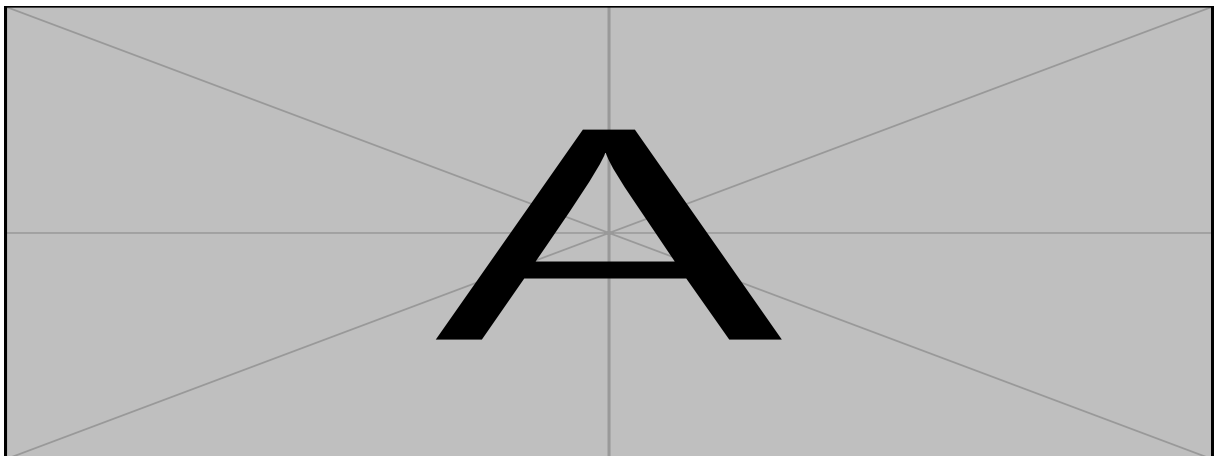


Figure 4.8: Diagram showing effective purpose of *inner loop*.

4.7.1 Constraint Formulation

We take the non-linear problem formulation from section 4.4, which contained ratios such as $\sigma_s = R_s/R$, and transform it into an entirely linear program by introducing auxiliary variables and linear constraints. In parallel, we incorporate the “range” abstraction from subsection 4.5.3 so that instead of summing over every sequence length, we only sum over a small set of ranges J . We also modify the problem to contain assignment variables (x values) for both prefill and decode. The result is a single, linearised load-balancing problem that jointly optimises prefill and decode throughput.

Linearisation Steps

In the original non-linear problem, each range’s achieved share σ_j involved dividing by the total throughput R , ($\sigma_j = R_j/R$). We avoid division by introducing one auxiliary variable per range and enforcing deviations through linear inequalities:

$$R_j - p_j R \leq \delta_j, \quad p_j R - R_j \leq \delta_j, \quad \delta_j \leq \tau p_j R,$$

for $R_j = R_j^{\text{prefill}}$ or R_j^{decode} and $R = R_{\text{prefill}}$ or R_{decode} .

Whenever an absolute deviation $|R_j - p_j R|$ appears in the original objective, we replace it by a nonnegative slack variable δ_j and two linear inequalities, ensuring $\delta_j \geq |R_j - p_j R|$. By defining R_j^{prefill} , R_j^{decode} , R_{prefill} , and R_{decode} as linear combinations of the x -variables, all throughput expressions become linear functions of the decision variables. Finally, to prevent zero-throughput solutions, we add linear lower-bounds $R_{\text{prefill}} \geq \varepsilon$, $R_{\text{decode}} \geq \varepsilon$, where $\varepsilon = 0.01$ is a small positive constant.

Combining Prefill and Decode

Because we must serve both prefill and decode for each range j , we include separate variables $x_{\text{prefill},ij}/x_{\text{decode},ij}$ and corresponding per-range throughput variables $R_j^{\text{prefill}}/R_j^{\text{decode}}$. We also add a δ_{match} variable to encourage equal balancing of throughput across both prefill and decode which is enforced by another pair of linear inequalities per range. so that $\delta_{\text{match},j} \geq |R_j^{\text{decode}} - R_j^{\text{prefill}}|$. By penalising these matching deviations in the objective, we ensure a balance between prefill and decode stages.

We also ensure each island is either performing prefill or decode by introducing a binary variable $z_i \in \{0, 1\}$ per island. Along with the constraints: $x_{\text{prefill},ij} \leq 1 - z_i$, $x_{\text{decode},ij} \leq z_i$, $\forall i \in I$, $j \in J$, so that if $z_i = 1$, the island’s $x_{\text{prefill},ij}$ are forced to zero (decode-only), and if $z_i = 0$, the island’s $x_{\text{decode},ij}$ are zero (prefill-only).

Input Data

I	(set of GPU islands),
J	(set of ranges, indexed by j),
b_{ij}^{prefill}	(measured prefill throughput of island i on range j),
b_{ij}^{decode}	(measured decode throughput of island i on range j),
p_j	(target probability of range j under workload w), $\sum_{j \in J} p_j = 1$,
τ	(maximum allowable per-range deviation factor),
M	(large constant for scaling deviation weights),
$\lambda_{\text{dev},j}$	$= \frac{M}{1 + p_j}$ (trade-off weight for deviations in range j).
ε	$= 0.01$ (minimum allowable throughput).

Decision Variables

$$\begin{aligned}
x_{\text{prefill},ij} &\geq 0, \quad \forall i \in I, j \in J, \quad (\text{fraction of island } i\text{'s capacity to prefill range } j); \\
x_{\text{decode},ij} &\geq 0, \quad \forall i \in I, j \in J, \quad (\text{fraction of island } i\text{'s capacity to decode range } j); \\
z_i &\in \{0, 1\}, \quad \forall i \in I, \quad (z_i = 1 \text{ if island } i \text{ is assigned to decode, else prefill}).
\end{aligned}$$

Intermediate Quantities

- **Per-range throughputs:**

$$R_j^{\text{prefill}} = \sum_{i \in I} b_{ij}^{\text{prefill}} x_{\text{prefill},ij}, \quad R_j^{\text{decode}} = \sum_{i \in I} b_{ij}^{\text{decode}} x_{\text{decode},ij}, \quad \forall j \in J.$$

- **Total throughputs:**

$$R_{\text{prefill}} = \sum_{j \in J} R_j^{\text{prefill}}, \quad R_{\text{decode}} = \sum_{j \in J} R_j^{\text{decode}}.$$

- **Per-range deviations:**

$$\delta_{\text{prefill},j} \geq 0, \quad \delta_{\text{decode},j} \geq 0, \quad \delta_{\text{match},j} \geq 0, \quad \forall j \in J.$$

- **Total deviation:**

$$D_{\text{total}} = \sum_{j \in J} (\delta_{\text{decode},j} + \delta_{\text{prefill},j} + \delta_{\text{match},j}).$$

Constraints

1. Each island allocates exactly one unit of capacity across prefill/decode and ranges:

$$\sum_{j \in J} (x_{\text{prefill}, ij} + x_{\text{decode}, ij}) = 1 \quad \forall i \in I.$$

2. Exclusive prefill/decode assignment per island:

$$x_{\text{prefill}, ij} \leq 1 - z_i, \quad x_{\text{decode}, ij} \leq z_i, \quad \forall i \in I, j \in J.$$

3. Enforce a minimum performance value:

$$R_{\text{prefill}} \geq \varepsilon, \quad R_{\text{decode}} \geq \varepsilon.$$

4. Deviation constraints for decode (per range j):

$$\begin{aligned} R_j^{\text{decode}} - p_j R_{\text{decode}} &\leq \delta_{\text{decode}, j}, \\ p_j R_{\text{decode}} - R_j^{\text{decode}} &\leq \delta_{\text{decode}, j}, \quad \forall j \in J. \\ \delta_{\text{decode}, j} &\leq \tau p_j R_{\text{decode}}, \end{aligned}$$

5. Deviation constraints for prefill (per range j):

$$\begin{aligned} R_j^{\text{prefill}} - p_j R_{\text{prefill}} &\leq \delta_{\text{prefill}, j}, \\ p_j R_{\text{prefill}} - R_j^{\text{prefill}} &\leq \delta_{\text{prefill}, j}, \quad \forall j \in J. \\ \delta_{\text{prefill}, j} &\leq \tau p_j R_{\text{prefill}}, \end{aligned}$$

6. Match throughput between prefill and decode (per range j):

$$\begin{aligned} R_j^{\text{decode}} - R_j^{\text{prefill}} &\leq \delta_{\text{match}, j}, \\ R_j^{\text{prefill}} - R_j^{\text{decode}} &\leq \delta_{\text{match}, j}, \quad \forall j \in J. \end{aligned}$$

7. Total deviation across all ranges:

$$\sum_{j \in J} (\delta_{\text{decode}, j} + \delta_{\text{prefill}, j} + \delta_{\text{match}, j}) = D_{\text{total}}.$$

Objective

$$\max \left(R_{\text{decode}} + R_{\text{prefill}} - \sum_{j \in J} \lambda_{\text{dev}, j} (\delta_{\text{decode}, j} + \delta_{\text{prefill}, j} + \delta_{\text{match}, j}) \right).$$

4.7.2 Implementation

We implement this constraint optimisation problem using **pulp**, a linear and mixed integer programming modeller written in Python. Given our problem uses both integer (z assignment variables) and float (x load variables), we need to use a Mixed-Integer Linear Programming Solver (MILP). The mixed nature worsens the performance but allows for the dynamic selection of which islands are provisioned for prefill and decode.

4.7.3 Optimisation

Given the *inner loop* will be executed several times by the *outer loop* during optimisation, we can improve performance through caching various results.

First, we can cache the **Formulate Ranges** step by recording the computed range values in a dictionary with the workload w and bin width. For each execution of *inner loop*, these values will be constant so caching prevents unnecessary re-computation.

Second we can cache the **Prefill Configuration Generation & Benchmark** process for islands of the same type and size. For a constant workload, we can expect the prefill configuration and benchmark output to be constant for the same given the same island size and type. The proposed island size-type combination will likely repeat several hundred times in the optimisation process, saving significant compute time.

Third, we can complete the same caching for the **Decode Configuration Generation & Benchmark** step. This step is executed during every iteration, like prefill, so we can expect equivalent performance gains.

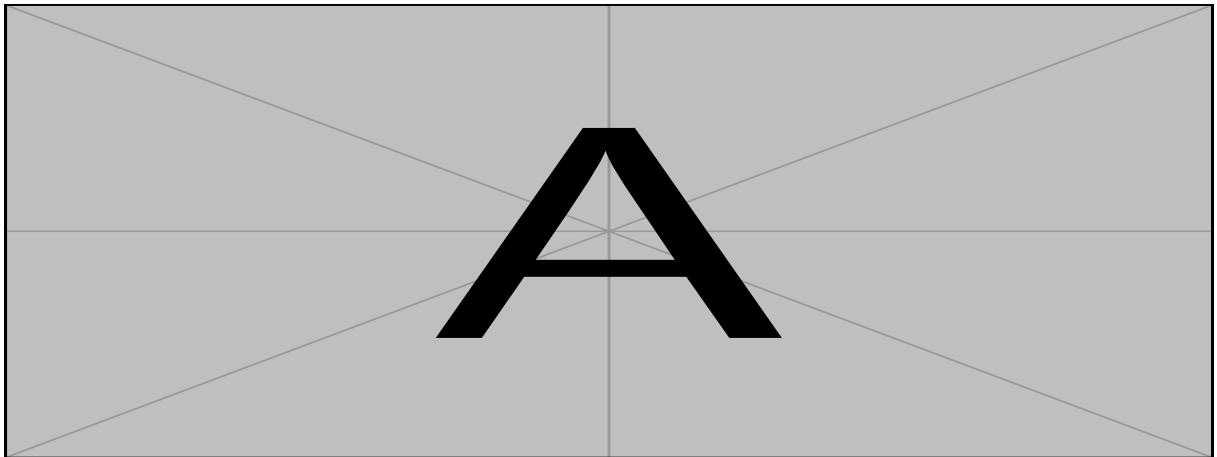


Figure 4.9: Diagram or graph showing effect of caching.

4.8 Outer Solver

The *outer loop* receives an input inventory, containing a list of GPU types ($t \in T$) and quantities (n_t), represented with the set $\{I_t : t \in T\}$. Given the inventory, the *outer loop's* job is to propose new island layouts ($\mathcal{I}$), which can be subsequently evaluated by the *inner loop*.

The *inner loop* execution is somewhat expensive, hence a complete grid search would incur significant compute time. Instead we use a refined Bayesian Optimisation search method with two different methods for searching, *direct* (subsection 4.8.1 and *divided* (subsection 4.8.2).

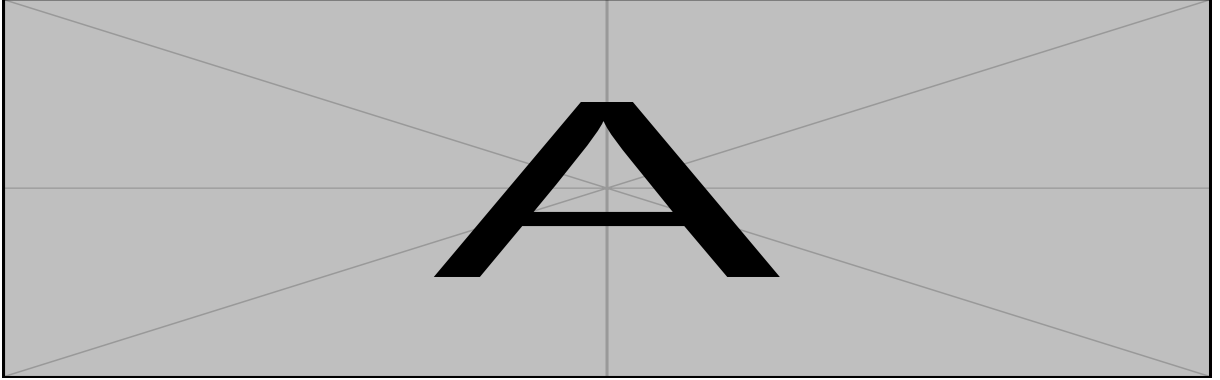


Figure 4.10: Diagram showing effective purpose of *outer loop*.

We select Bayesian Optimisation specifically due to its ability to efficiently explore high-dimensional, expensive-to-evaluate spaces with only a few hundred evaluations, rather than the exponential cost of a full grid search. By maintaining a surrogate model of the island-layout performance (Gaussian process over layouts), Bayesian Optimisation balances exploration (sampling layouts in poorly understood regions) with exploitation (refining around the current best layouts).

Unlike reinforcement learning, which often requires thousands of interactions to learn a policy, BO is designed for “one-shot” black-box optimisation. With the correct parameters it can deliver near-optimal solutions with far fewer simulator calls, minimal hyper-parameter tuning, and strong convergence.

4.8.1 Direct Island Generation

In the *direct* island generation approach, islands are explicitly defined within the Bayesian Optimisation process. Configurations correspond to a fixed number of islands, with each candidate solution representing a specific allocation strategy. The number of GPUs in each island is directly part of the output space, where specific variables represent the GPU count per island ‘slot’. This setup provides precise control over how GPUs are grouped, but it also requires well-defined constraints to ensure that all generated configurations are valid.

Search Parameters

- T : the set of GPU types, e.g., $T = \{\text{A100}, \text{V100}, \dots\}$
- K : the number of GPU slots per type
- $x_{t,k}$: the integer decision variable representing the number of GPUs of type $t \in T$ assigned to slot $k \in \{1, \dots, K\}$
- I_t : the total inventory of GPU type $t \in T$

Search Constraints

For each GPU type $t \in T$, we enforce that the total number of GPUs allocated across all K slots does not exceed the inventory I_t :

$$\sum_{k=1}^K x_{g,k} \leq I_t, \quad \forall t \in T$$

Implementation

- **Ax Implementation.**
- **Why SAASBO may be required (high dimensionality)**
- **Why this approach is challenging.**

4.8.2 Divider Island Generation

The *divider* island generation method uses a more abstract representation that substantially reduces the dimensionality of the BO problem. Explicit allocation at the slot level requires one parameter per slot per GPU type, resulting in a parameter space of size $|G| \times K$. This approach becomes quickly computationally expensive due to the high number of search parameters and the need for constraints to ensure valid configurations.

To address this, the divider-based approach reduces the search space to just two parameters per GPU type: the total number of islands and a scalar that controls the balance of GPU allocation across those islands. This formulation results in a significantly smaller parameter space of size $|G| \times 2$, enabling more efficient optimisation and faster convergence.

An advantage of this method is that it avoids the need for explicit resource constraints, which are otherwise necessary to enforce inventory limits and prevent invalid configurations. By shifting from low-level allocation to higher-level distribution strategies, divider island generation maintains control over performance-influencing characteristics while offering a more scalable and interpretable optimisation process.

Input Variables

Let G_{total} denote the total number of GPUs available for that type, and let N_{target} represent the desired number of islands to create. We can allow the actual number of islands may be lower if there are not enough GPUs to satisfy the minimum-size requirement. The parameter S_{skew} controls how any leftover GPUs (after each island has been assigned its minimum) are distributed: when $S_{\text{skew}} \approx 0.0$, the extra GPUs are spread as evenly as possible; if $S_{\text{skew}} > 0.0$, islands with higher indices receive proportionally more of the remainder; and if $S_{\text{skew}} < 0.0$, islands with lower indices receive more. Finally, I_{min} specifies the minimum number of GPUs that any island must contain.

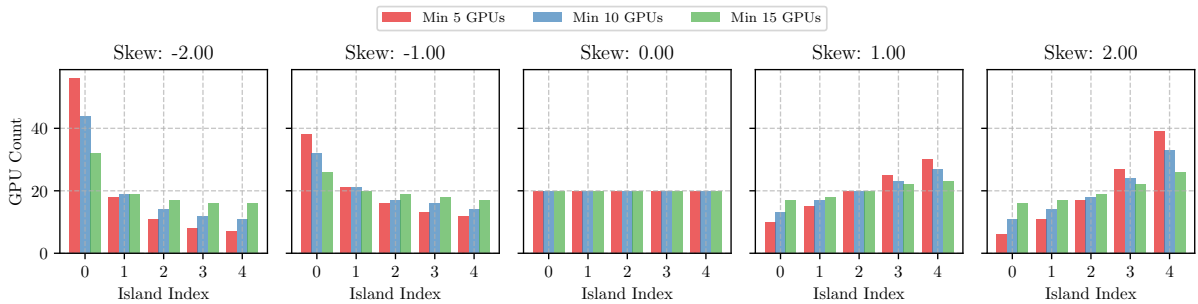


Figure 4.11: Graph displaying the effect of changing the S_{skew} value where $G_{\text{total}} = 100$ and $N_{\text{target}} = 5$.

Preconditions

Initially, we verify that the input parameters meet a series of constraints, if any of these conditions is not met, the procedure raises an error and terminates immediately. We require $I_{\min} > 0$, G_{total} and N_{target} must be nonnegative. Next, we perform trivial checks to assess whether it is possible to create at least one island given the input parameters. If $N_{\text{target}} = 0$, $G_{\text{total}} = 0$ or $G_{\text{total}} < I_{\min}$, then no islands are to be created and the function returns the empty set $\emptyset$. Similarly, if , there are insufficient GPUs to form even a single island of size $I_{\min}$, so the function again returns $\emptyset$.

Number of Islands

To establish how many islands can feasibly be created given the available GPUs and the minimum-size requirement, we first compute $N_{\text{max_possible}} = \lfloor G_{\text{total}}/I_{\min} \rfloor$, which represents the maximum number of islands if each island is allocated exactly $I_{\min}$ GPUs. We then set $N_{\text{actual}} = \min(N_{\text{target}}, N_{\text{max_possible}})$, so that the actual number of islands cannot exceed either the user's target or the physical limit imposed by the GPU pool. If $N_{\text{actual}} = 0$, this indicates that even a single island of size $I_{\min}$ cannot be formed (either because $N_{\text{target}} = 0$ or because $G_{\text{total}} < I_{\min}$), and the function returns the empty set $\emptyset$. Otherwise, N_{actual} islands will be created, each initially allocated $I_{\min}$ GPUs before any further distribution of the remaining GPUs occurs.

Base Allocation

In the base allocation step, we allocate $I_{\min}$ GPUs to each of the N_{actual} islands, so that the total GPUs used for this initial distribution is $G_{\text{base_used}} = N_{\text{actual}} \cdot I_{\min}$. The number of GPUs remaining to be distributed is then $G_{\text{remaining}} = G_{\text{total}} - G_{\text{base_used}}$. At this point, every island has been provisioned with its minimum allocation of $I_{\min}$ GPUs, and the remaining $G_{\text{remaining}}$ GPUs will be allocated in subsequent steps.

Island Distribution

In the case where $G_{\text{remaining}} = 0$, all GPUs have already been assigned to the N_{actual} islands at their minimum size, so we make no weighted adjustments. When $N_{\text{actual}} = 1$, there is only a single island so we add all $G_{\text{remaining}}$ GPUs to that island. Otherwise we can distribute the remaining GPUs using the S_{skew} value.

We first compute a weight w_k for each island k , where k ranges from 1 to N_{actual} , computed as $w_k = k^{S_{\text{skew}}}$. However, if S_{skew} is numerically close to zero, each island is given an equal weight of $w_k = 1.0$. These weights are then normalised so that we can derive proportions p_k , indicating each island's share of the remaining GPUs. The sum of the weights is computed as

$$W_{\text{sum}} = \sum_{j=1}^{N_{\text{actual}}} w_j.$$

Each proportion is determined by dividing the weight of an island by the weight sum:

$$p_k = \frac{w_k}{W_{\text{sum}}}.$$

Using these proportions, we compute the ideal (possibly fractional) allocation s_k^{ideal} for each island:

$$s_k^{\text{ideal}} = p_k \cdot G_{\text{remaining}}.$$

Integer Adjustment

Because GPUs can only be allocated in integer units, we use the largest remainder method to convert the fractional allocations into integers. First, each island k is provisionally assigned a base number of GPUs:

$$s_k^{\text{base_int}} = \lfloor s_k^{\text{ideal}} \rfloor.$$

The current island sizes (initialised to I_{min}) are then increased by these integer allocations.

Search Parameters

- T : the set of GPU types, e.g., $T = \{\text{A100}, \text{V100}, \dots\}$
- n_t : the target number of islands for GPU type $t \in T$
- s_t : a vector value representing the skew or balance in GPU allocation across the n_t islands for GPU type t
- I_t : the total available inventory of GPU type t

These two parameters (n_t and s_t) define the entire resource distribution strategy, allowing the Bayesian Optimisation process to search over high-level allocation patterns while implicitly satisfying inventory constraints by design.

Implementation

- Ax Implementation.
- SAASBO not required.

Chapter 5

Evaluation

- Intro to Chapter.
- Key - compare heterogenous vs homogenous (show why heterogeneous is better)
- Overall system performance (key graphs only).

Talk about Experimental Setup!!!!

5.1 Aims

May include overall goals of the evaluation

5.2 System Performance

Some graphs to highlight the performance of the system

5.3 Inner Loop

We test the *inner loop* using four test cases (island layouts) shown in Table 5.1, which include two homogeneous scenarios and two heterogeneous scenarios. One scenario from both homogeneous and heterogeneous uses GPU islands of the same size, the other uses GPU islands of different sizes. These islands are crafted to show the importance of selecting appropriate island sizes and potential value of heterogeneous clusters.

Island	GPU Model	Sizes (Size $\times$ Count)	Cost
(a) Homogeneous Same Size	DGX-B300	32×8	—
(b) Homogeneous Different Sizes	DGX-B300	$32 \times 2, 16 \times 4, 8 \times 8, 4 \times 16$	—
(c) Heterogeneous Same Size	DGX-B300	32×2	—
	Rubin-NVL144	32×2	—
	H800	32×2	—
	H20	32×2	—
(d) Heterogeneous Different Sizes	DGX-B300	$32 \times 1, 16 \times 1, 8 \times 1, 4 \times 2$	—
	Rubin-NVL144	$32 \times 1, 16 \times 1, 8 \times 1, 4 \times 2$	—
	H200	$8 \times 4, 4 \times 8$	—
	H20	$8 \times 4, 4 \times 8$	—

Table 5.1: GPU island test cases for *inner loop* analysis, with two homogeneous scenarios and two heterogeneous scenarios

As the *inner loop* solution generation is deterministic we only execute each test case once, unless stated otherwise. This is because the linear solver will produce the exact same solution upon every execution, given the same input variables.

5.3.1 Solver Results

We first simultaneously select the optimum parallelism configuration and benchmark the provided GPU islands for both prefill and decode. The resultant throughput values are displayed in Figure 5.1, where we can see that every island class is able to contribute to prefill throughput, but not all islands are capable of decode. This is due to the decode phase requiring more memory to execute, hence not all islands are compatible.

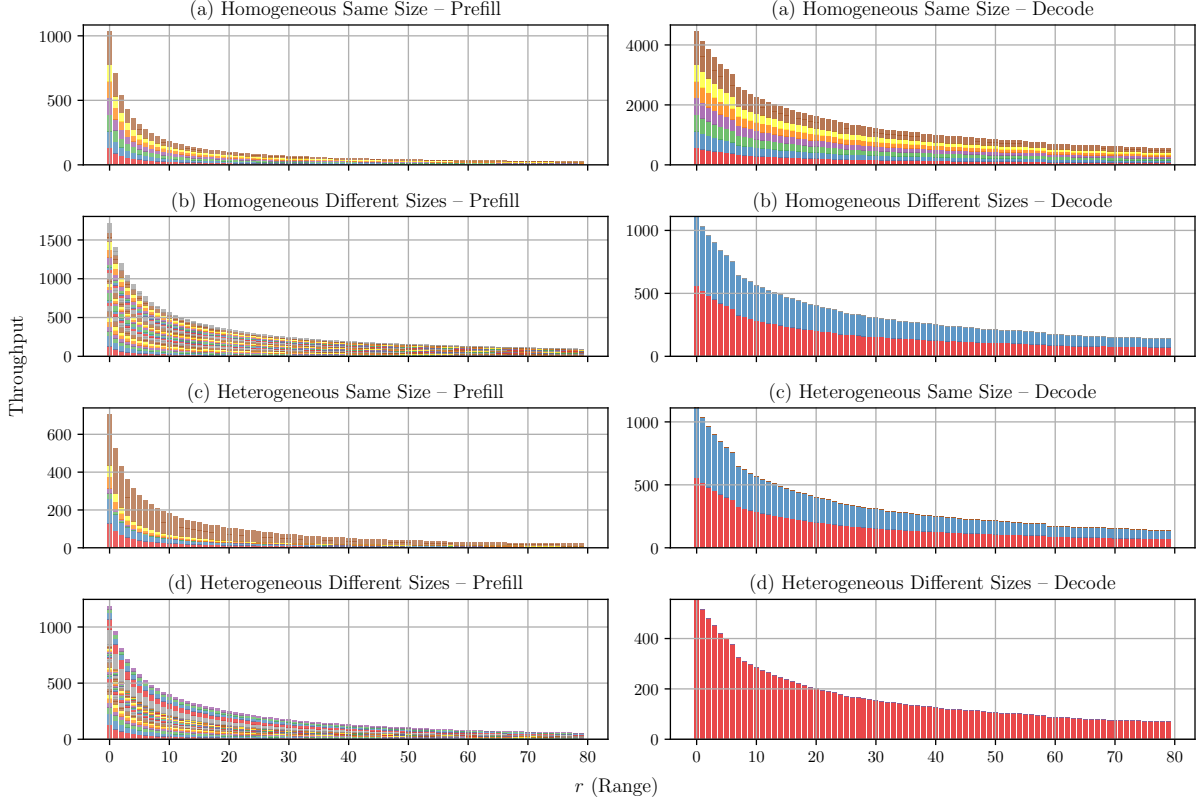


Figure 5.1: Benchmark results for every island test case

Using the collected benchmark values, we populate the b_{ij}^{prefill} and b_{ij}^{decode} variables in the linear solver. Then, using artificial workload as previously shown in Figure 4.1, we execute the solve function to find the optimum $x_{\text{prefill}, ij}$ and $x_{\text{decode}, ij}$ values. The resultant assignment percentages are displayed in Figure 5.2 for both the prefill and decode phase.

In the homogenous scenario, the assignment allocation percentages closely resemble the target workload. In contrast, heterogenous assignment allocation percentages appear random. This is to be expected, as it reflects the heterogeneous nature of island performance, where one island might be allocated few ranges due to its “slow” performance. Also, for all scenarios, one large island is generally allocated for decode and many islands for prefill, this displays that prefill is the bottleneck for throughput.

Collating the assignment percentages with throughout measurements (Figure 5.2 and Figure 5.1) we can plot the actual throughput per island, shown in Figure 5.3.

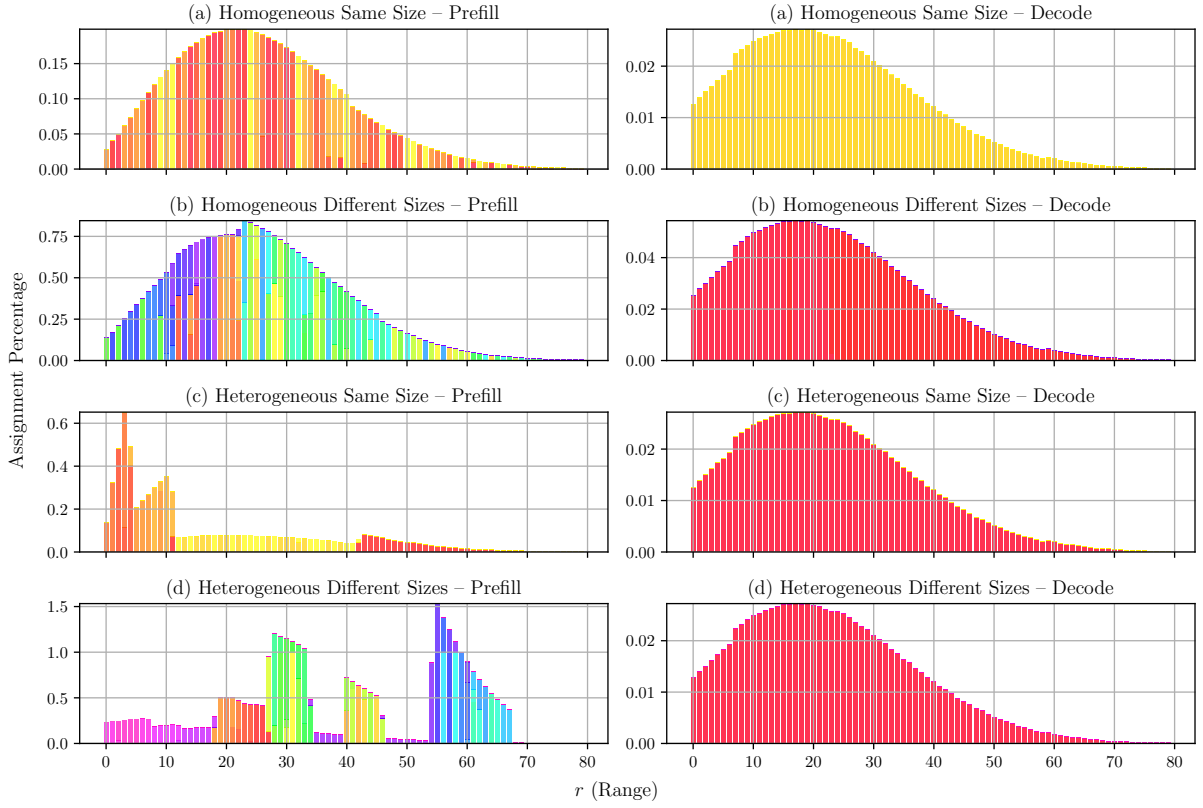


Figure 5.2: Assignment values percentage (per range) for each every island test case, using artificial workload as described in Figure 4.1

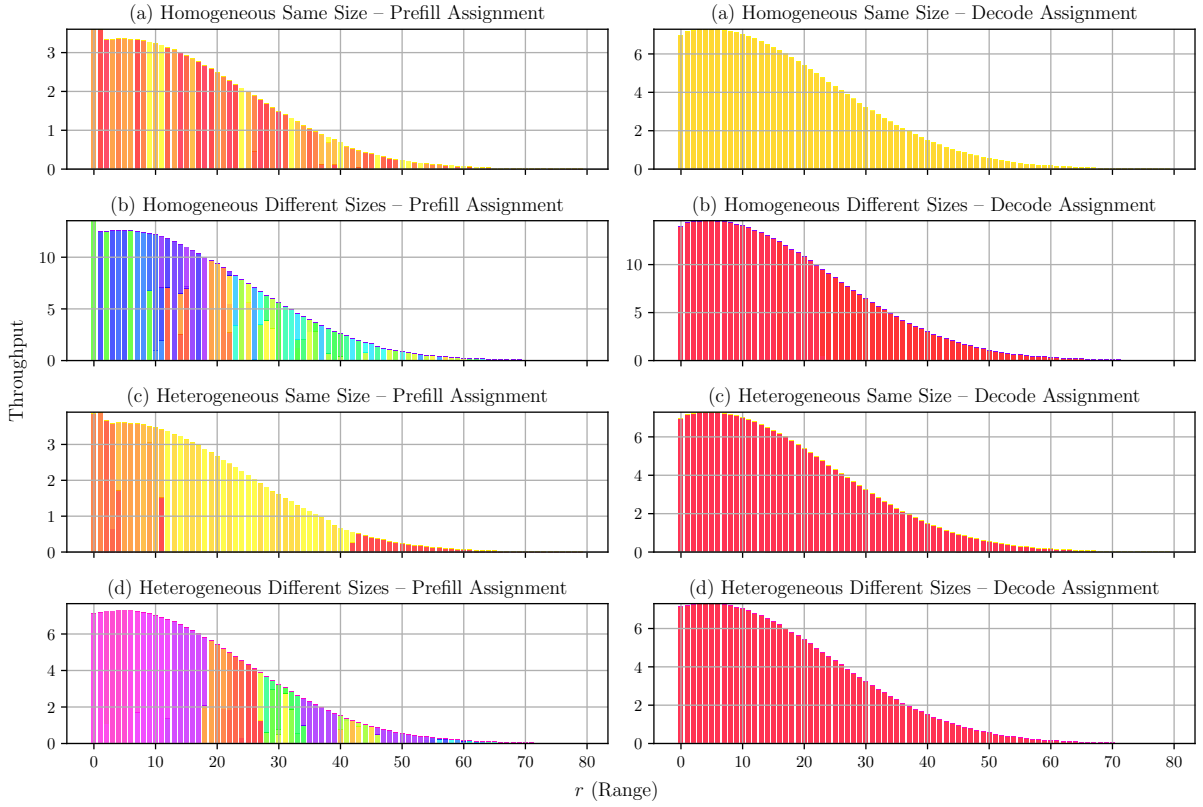


Figure 5.3: Throughput per range for each every island test case, using artificial workload as described in Figure 4.1

5.3.2 Evaluator Results

Previous plots demonstrate the allocation results using direct outputs from the linear solver at the specified range resolution. We need to evaluate the solutions using the complete expected input sequence length distribution $p(s | w)$ to highlight any differences in performance. These results are displayed in Figure 5.4, where *evaluated* refers to results from the evaluator and *model* refers to results directly from the solver. We also include the workload *trace* as a reference to what the target throughput distribution is.

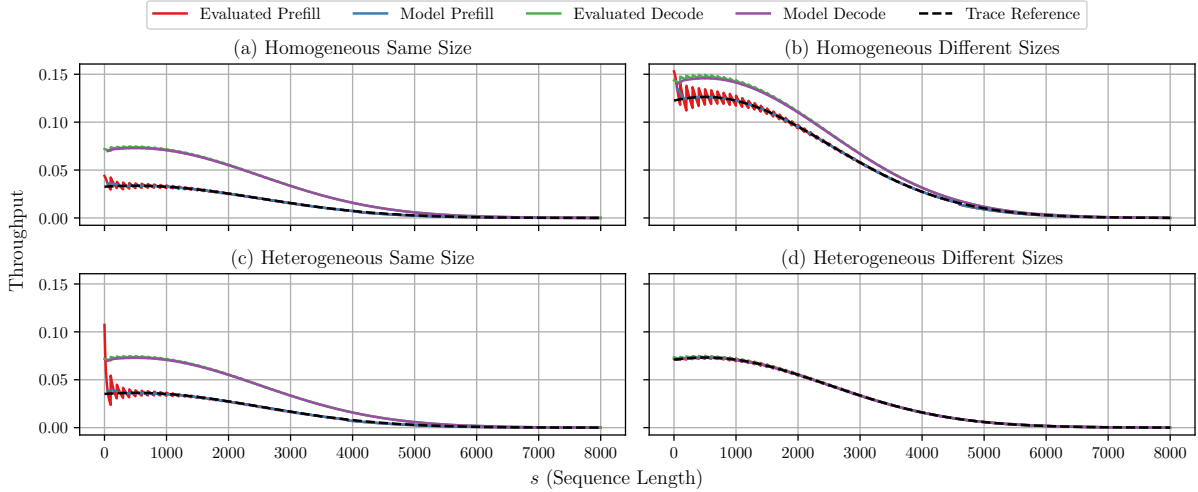


Figure 5.4: Inner Trace Throughput

Evaluator Method

The true throughput is calculated by first benchmarking each exact sequence length (whilst also selecting the best parallelism configuration), for each island using the same method as described in section 4.7. Then, the throughputs are weighted with the p value of each sequence length s along with the island percentage allocation x , of the range the sequence length resides within.

5.3.3 Resolution Impact

- To Complete.
- How resolution impacts solution quality vs time.

5.3.4 Objective Impact

- To Complete.
- How the objective function has been tuned.

5.4 Outer Loop (Divider)

- Test setup.
- Test approach (repeat 10 times).

5.4.1 Batch Size Analysis

- How batch size influences results (variance).
- Solution achieved in 20 iterations.

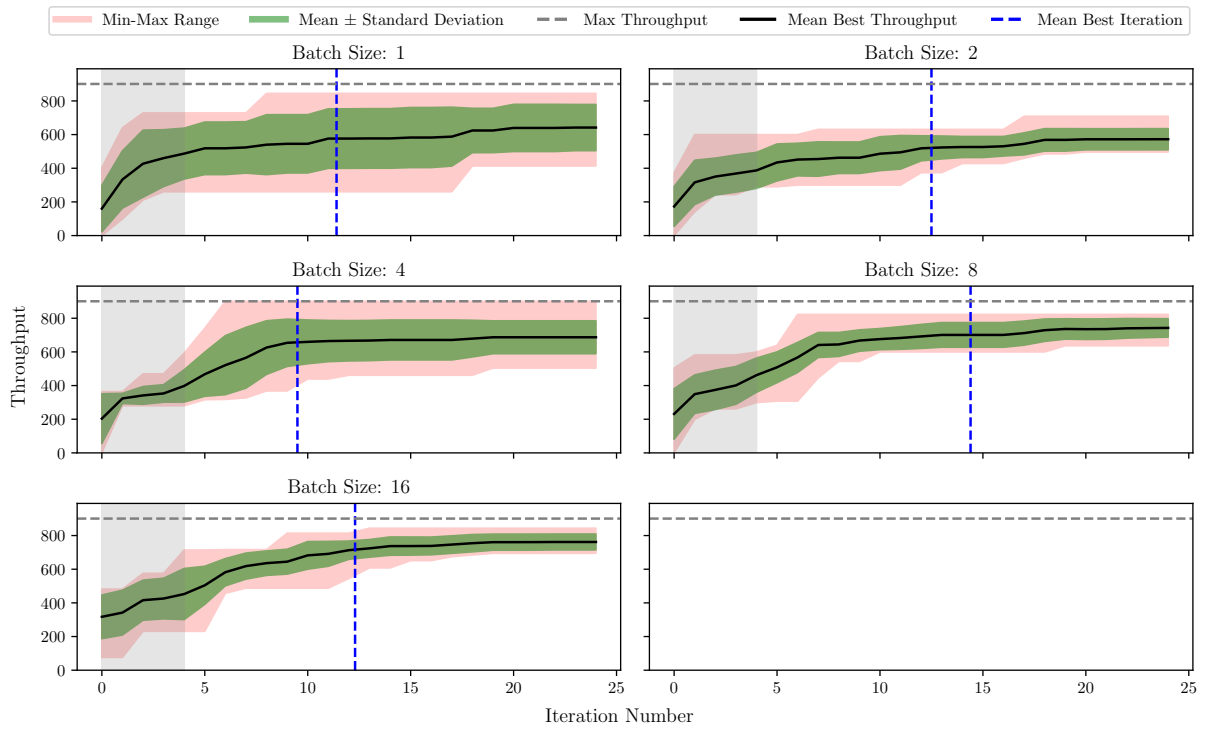


Figure 5.5: Batch Size Analysis

- How long on average to best solution.

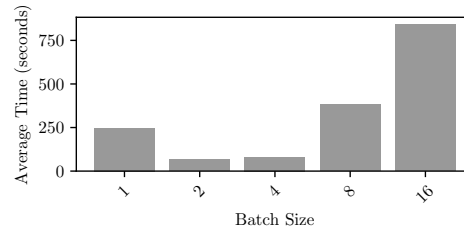


Figure 5.6: Batch Size Analysis (Time)

5.4.2 Skew Analysis

- How changing the search params (skew) affects solution.

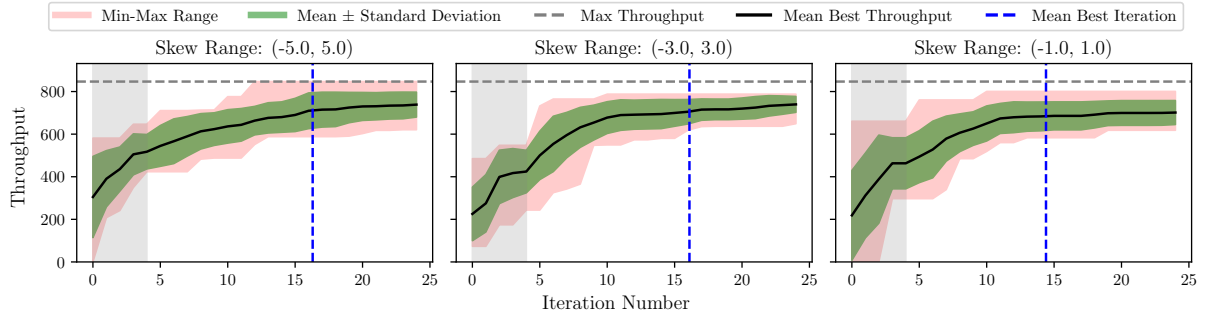


Figure 5.7: Skew Analysis

5.4.3 Number of Islands

- How changing the search params (number of islands) affects solution.

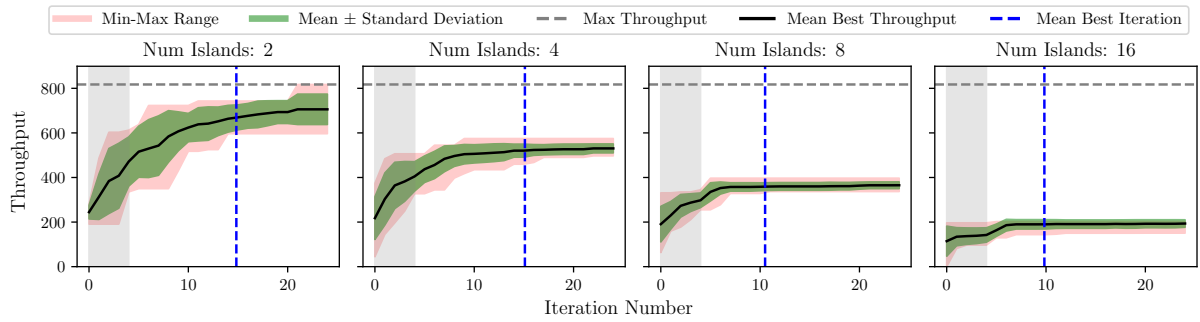


Figure 5.8: Min Island Analysis

5.5 Outer Loop (Direct)

- Test setup.
- Test approach (repeat 10 times).

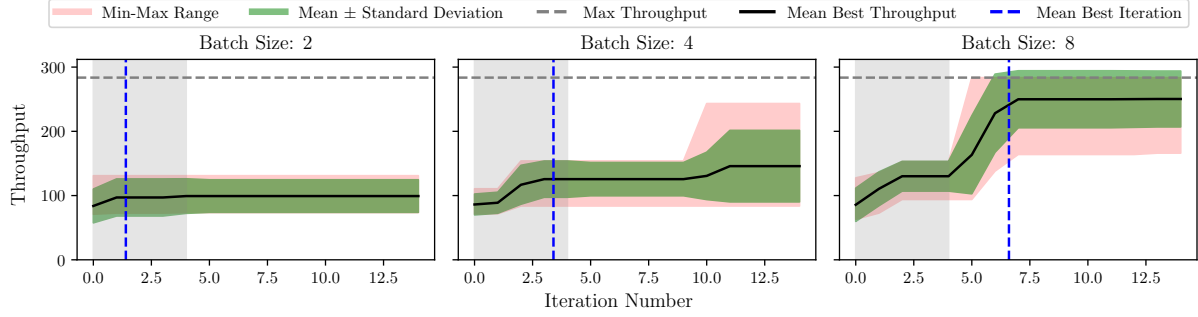


Figure 5.9: Direct SAASBO Batch Size Analysis

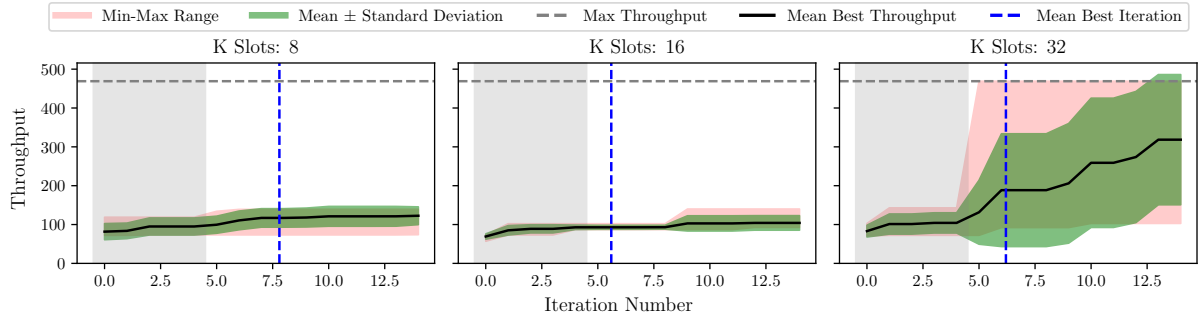


Figure 5.10: Direct SAASBO KSlots Size Analysis

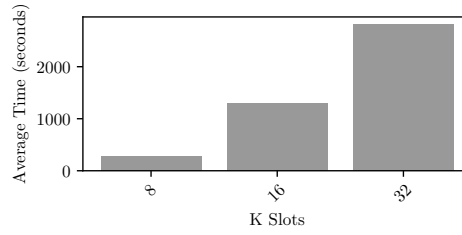


Figure 5.11: Direct SAASBO Batch Size Analysis (Time)

5.6 Real-World

- End-to-end analysis with real-world traces.

5.6.1 Traces

Azure LLM

Source [23]

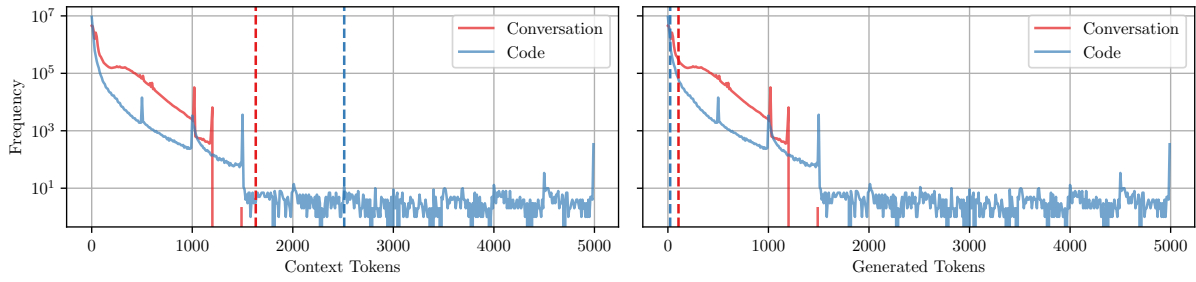


Figure 5.12: Azure LLM Context and Generated Tokens

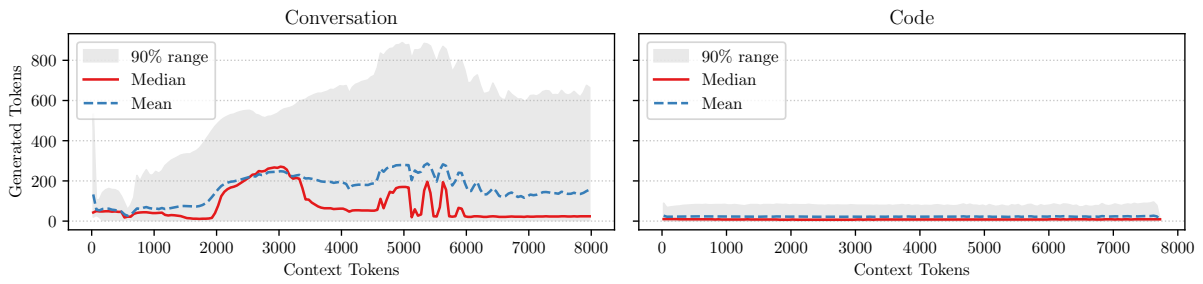


Figure 5.13: Azure LLM Context vs Generated Tokens

BurstGPT

Source [24]

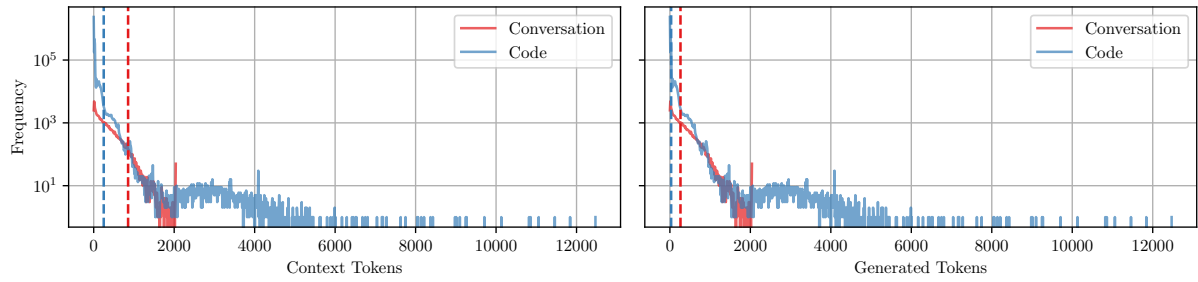


Figure 5.14: BurstGPT Context and Generated Tokens

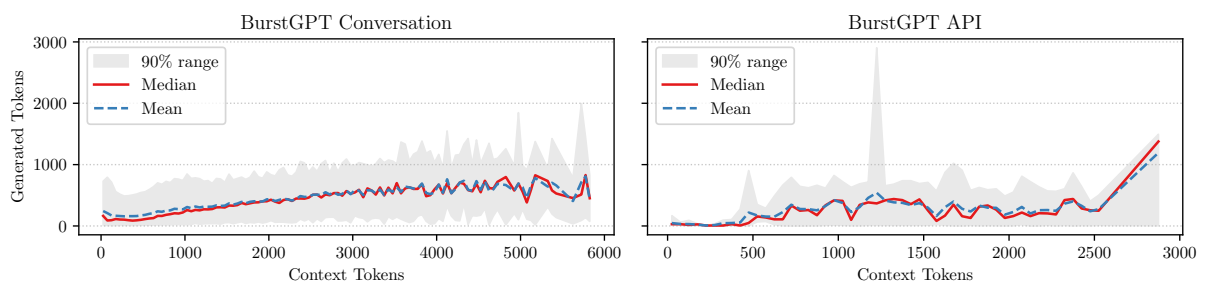


Figure 5.15: DBurstGPT Context vs Generated Tokens

5.6.2 Results

- How the traces perform.

5.7 Heterogeneous vs Homogenous

- Throughput vs Cost of Heterogeneous vs Homogenous cluster.

5.8 Discussion

(if required - may merge this with the heterogeneous vs homogenous)

Chapter 6

Conclusion

- Formulation success.
- Inner loop is great, speedy and more accurate (resolution).
- Outer loop is good, BO is pretty good
- Future, include cost into the BO loop, with dynamic inventory

Maybe a small summary too

6.1 Conclusion

Stuff.

6.2 Future Work

Stuff.

References

- [1] A. Vaswani *et al.* “Attention Is All You Need.” Comment: 15 pages, 5 figures. arXiv: 1706.03762 [cs]. (Aug. 2, 2023), <http://arxiv.org/abs/1706.03762> (visited on Jun. 7, 2025), pre-published.
- [2] M. Lewis *et al.* “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.” arXiv: 1910.13461 [cs]. (Oct. 29, 2019), <http://arxiv.org/abs/1910.13461> (visited on Jun. 8, 2025), pre-published.
- [3] J. Devlin *et al.* “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” arXiv: 1810.04805 [cs]. (May 24, 2019), <http://arxiv.org/abs/1810.04805> (visited on Jun. 7, 2025), pre-published.
- [4] A. Radford *et al.*, “Improving Language Understanding by Generative Pre-Training,”
- [5] Y. Sheng *et al.* “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU.” arXiv: 2303.06865 [cs]. (Jun. 12, 2023), <http://arxiv.org/abs/2303.06865> (visited on Jun. 8, 2025), pre-published.
- [6] “Mastering LLM Techniques: Inference Optimization,” NVIDIA Technical Blog. (Nov. 17, 2023), <https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/> (visited on Jun. 8, 2025).
- [7] P. Patel *et al.* “Splitwise: Efficient generative LLM inference using phase splitting.” Comment: 12 pages, 19 figures. arXiv: 2311.18677 [cs]. (May 20, 2024), <http://arxiv.org/abs/2311.18677> (visited on Jun. 3, 2025), pre-published.
- [8] R. Pope *et al.* “Efficiently Scaling Transformer Inference.” arXiv: 2211.05102 [cs]. (Nov. 9, 2022), <http://arxiv.org/abs/2211.05102> (visited on Jun. 8, 2025), pre-published.
- [9] N. Shazeer *et al.* “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.” arXiv: 1701.06538 [cs]. (Jan. 23, 2017), <http://arxiv.org/abs/1701.06538> (visited on Jun. 8, 2025), pre-published.
- [10] D. Lepikhin *et al.* “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.” arXiv: 2006.16668 [cs]. (Jun. 30, 2020), <http://arxiv.org/abs/2006.16668> (visited on Jun. 8, 2025), pre-published.

- [11] Y. Zhong *et al.* “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving.” Comment: OSDI 2024. arXiv: 2401.09670 [cs]. (Jun. 6, 2024), <http://arxiv.org/abs/2401.09670> (visited on Jun. 3, 2025), pre-published.
- [12] “ShareGPT.” (), <https://sharegpt.com> (visited on Jun. 7, 2025).
- [13] M. Chen *et al.* “Evaluating Large Language Models Trained on Code.” Comment: corrected typos, added references, added authors, added acknowledgements. arXiv: 2107.03374 [cs]. (Jul. 14, 2021), <http://arxiv.org/abs/2107.03374> (visited on Jun. 7, 2025), pre-published.
- [14] Y. Bai *et al.* “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.” Comment: ACL 2024. arXiv: 2308.14508 [cs]. (Jun. 19, 2024), <http://arxiv.org/abs/2308.14508> (visited on Jun. 7, 2025), pre-published.
- [15] W. Kwon *et al.* “Efficient Memory Management for Large Language Model Serving with PagedAttention.” Comment: SOSP 2023. arXiv: 2309.06180 [cs]. (Sep. 12, 2023), <http://arxiv.org/abs/2309.06180> (visited on Jun. 7, 2025), pre-published.
- [16] *Deepspeedai/DeepSpeed*, deepspeedai, Jun. 7, 2025. <https://github.com/deepspeedai/DeepSpeed> (visited on Jun. 7, 2025).
- [17] Y. Jiang *et al.* “ThunderServe: High-performance and Cost-efficient LLM Serving in Cloud Environments.” Comment: MLSys 2025. arXiv: 2502.09334 [cs]. (Feb. 13, 2025), <http://arxiv.org/abs/2502.09334> (visited on May 13, 2025), pre-published.
- [18] Y. Jiang *et al.* “HexGen: Generative Inference of Large Language Model over Heterogeneous Environment.” Comment: Accepted by ICML 2024. arXiv: 2311.11514 [cs]. (May 27, 2024), <http://arxiv.org/abs/2311.11514> (visited on Nov. 18, 2024), pre-published.
- [19] DeepSeek-AI *et al.* “DeepSeek-V3 Technical Report.” arXiv: 2412.19437 [cs]. (Feb. 18, 2025), <http://arxiv.org/abs/2412.19437> (visited on Jun. 7, 2025), pre-published.
- [20] A. Agrawal *et al.* “Vidur: A Large-Scale Simulation Framework For LLM Inference.” arXiv: 2405.05465 [cs]. (May 21, 2024), <http://arxiv.org/abs/2405.05465> (visited on Nov. 19, 2024), pre-published.
- [21] icezack12, *Icezack12/shallowsim*, Jun. 3, 2025. <https://github.com/icezack12/shallowsim> (visited on Jun. 3, 2025).
- [22] H. Zhang *et al.* “A Hardware Evaluation Framework for Large Language Model Inference.” arXiv: 2312.03134 [cs]. (Dec. 5, 2023), <http://arxiv.org/abs/2312.03134> (visited on Jun. 3, 2025), pre-published.

- [23] J. Stojkovic *et al.* “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency.” arXiv: 2408.00741 [cs]. (Aug. 1, 2024), <http://arxiv.org/abs/2408.00741> (visited on Jun. 3, 2025), pre-published.
- [24] Y. Wang *et al.* “BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems.” arXiv: 2401.17644 [cs]. (May 26, 2025), <http://arxiv.org/abs/2401.17644> (visited on Jun. 3, 2025), pre-published.